

WEEK 4

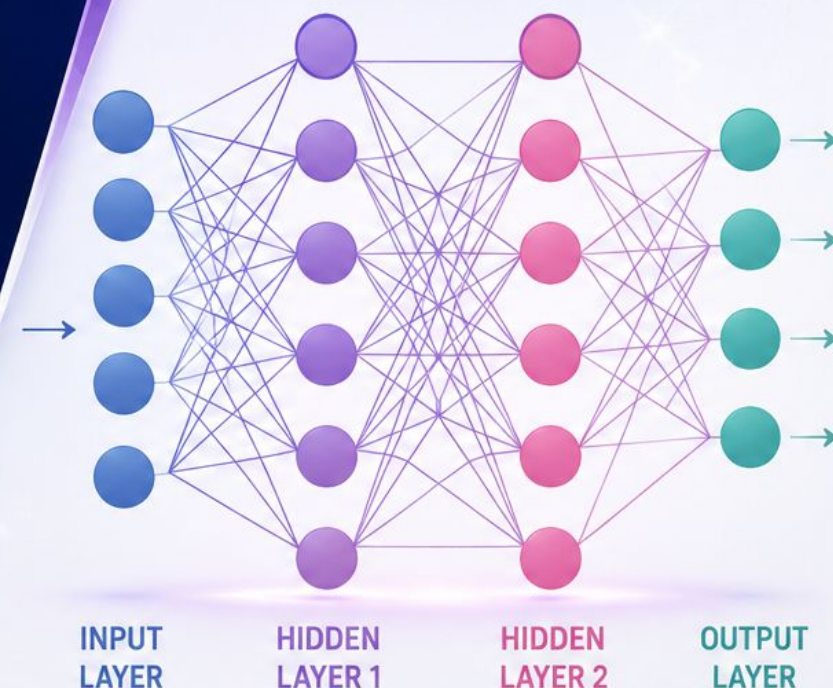
# NEURAL NETWORKS FOR NLP

Building the foundation for modern  
Natural Language Processing



## TODAY WE WILL EXPLORE:

- ✓ Foundations of Neural Networks
- ✓ How Neural Networks learn
- ✓ Applying Neural Networks to NLP tasks
- ✓ Limitations and what comes next



text prediction  
**language learning** embeddings  
representation context  
understanding



Understand how  
neural networks work



Learn how networks  
make predictions



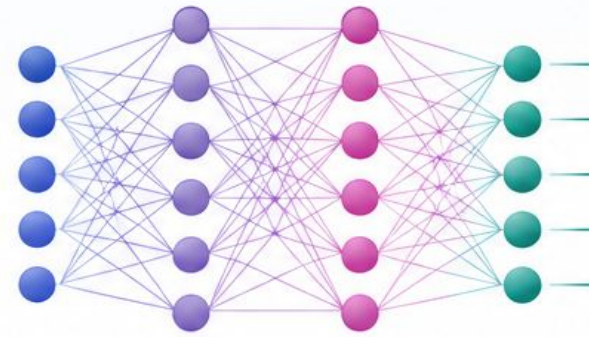
See how they are used  
in NLP applications



Prepare for advanced  
NLP models

## LEARNING OUTCOMES

By the end of this week, you will be able to:



1



### Explain the role of neural networks in NLP

Describe why neural networks are effective for learning patterns from text data and how they differ from traditional machine learning models.

2



### Understand the building blocks of neural networks

Explain the structure of a neuron, activation functions, and how inputs are transformed to produce an output.

3



### Describe the structure of a neural network

Identify input, hidden, and output layers and explain how information flows forward to make predictions.

4



### Understand how neural networks learn

Explain loss, backpropagation, and gradient descent at a conceptual level and how weights are updated during training.

5



### Apply neural networks to NLP tasks

Use word embeddings as inputs to build a simple neural network for text classification (e.g., sentiment analysis).

6



### Evaluate model performance

Interpret evaluation metrics and understand the concepts of training, validation, and test sets.

7



### Identify common training challenges

Explain overfitting and underfitting and apply techniques such as dropout to improve generalization.

8



### Recognize the limitations of basic neural networks

Understand why feedforward networks have limited ability to capture sequential information and what advanced models address this limitation.



### BIG GOAL:

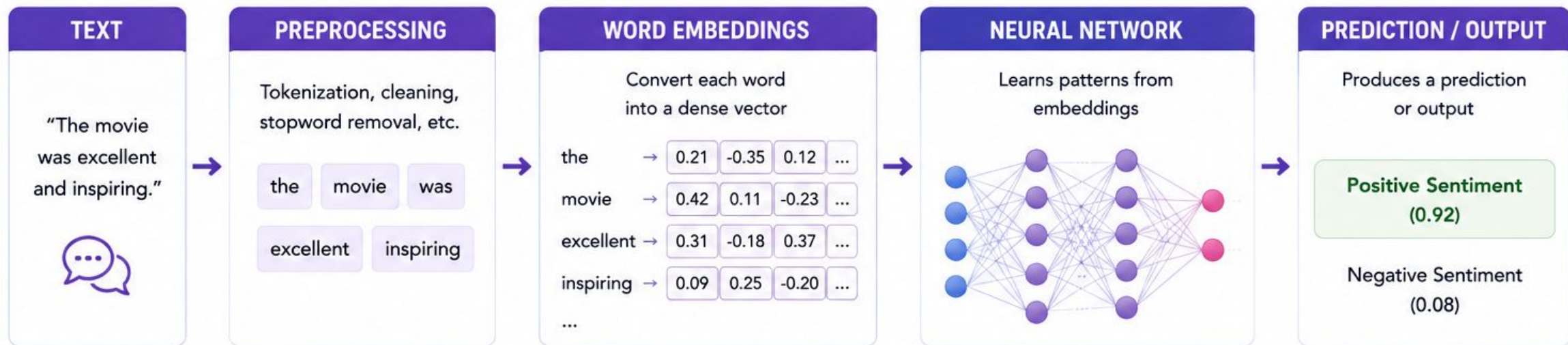
Build a strong foundation in neural networks so we can apply them effectively to real-world NLP problems.



# From Word Embeddings to Neural Networks



In Week 3, we converted text into numerical representations.  
In Week 4, we learn how neural networks use these representations to understand patterns and make predictions.



## KEY MESSAGE

Neural networks take numerical representations (especially word embeddings) as inputs, learn complex patterns from data, and produce meaningful predictions for NLP tasks such as classification, sentiment analysis, and more.



# Why Use Neural Networks for NLP?

Neural networks are especially powerful for NLP because they can learn complex patterns from data automatically and improve as more data becomes available.



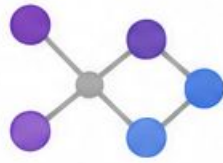
## Learn Complex Patterns Automatically

Automatically learn hidden patterns and relationships in text data without manual feature engineering.



## Capture Semantic Meaning

Understand context, meanings and relationships between words beyond simple counts or rules.



## Work with Dense Representations

Effectively use word embeddings and other dense representations that capture richer semantic information.



## High Performance on Many Tasks

Achieve state-of-the-art results on a wide range of NLP tasks (local and international).



## Scalable with More Data

Improve continuously as more data becomes available; handle large and complex datasets effectively.



## Generalize to New Data

Learn representations that generalize better to unseen text and new situations.



## KEY TAKEAWAY

Neural networks are well-suited for NLP because they can learn from raw data, capture meaningful patterns, and make accurate predictions across diverse language tasks.



# Traditional Machine Learning vs Neural Networks for NLP



Both approaches can solve NLP problems, but neural networks are better at learning from raw text and capturing complex patterns automatically.

| Aspect                                                                                                                | Traditional Machine Learning (for NLP)                                     | Neural Networks (for NLP)                                                  |
|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------|
|  Feature Representation              | Requires manual feature engineering (e.g., Bag-of-Words, TF-IDF, n-grams). | Learns features automatically from raw text or word embeddings.            |
|  Ability to Capture Complex Patterns | Limited ability to capture complex patterns and long-range dependencies.   | Excellent at capturing complex patterns, context, and relationships.       |
|  Data Requirement                    | Often works with smaller datasets.                                         | Typically needs more data to perform well, but scales and keeps improving. |
|  Performance                         | Lower performance on complex NLP tasks (e.g., translation, summarization). | State-of-the-art performance on most NLP tasks.                            |
|  Handling Unstructured Data         | Struggles with unstructured text; needs heavy preprocessing.               | Designed to handle unstructured text more effectively.                     |
|  Flexibility                       | Needs different features/models for different tasks.                       | More flexible; the same model architecture can be adapted to many tasks.   |
|  Example Algorithms / Models       | Naïve Bayes, SVM, Logistic Regression, Random Forest.                      | Feedforward NN, CNN, RNN, LSTM, GRU, Transformers.                         |

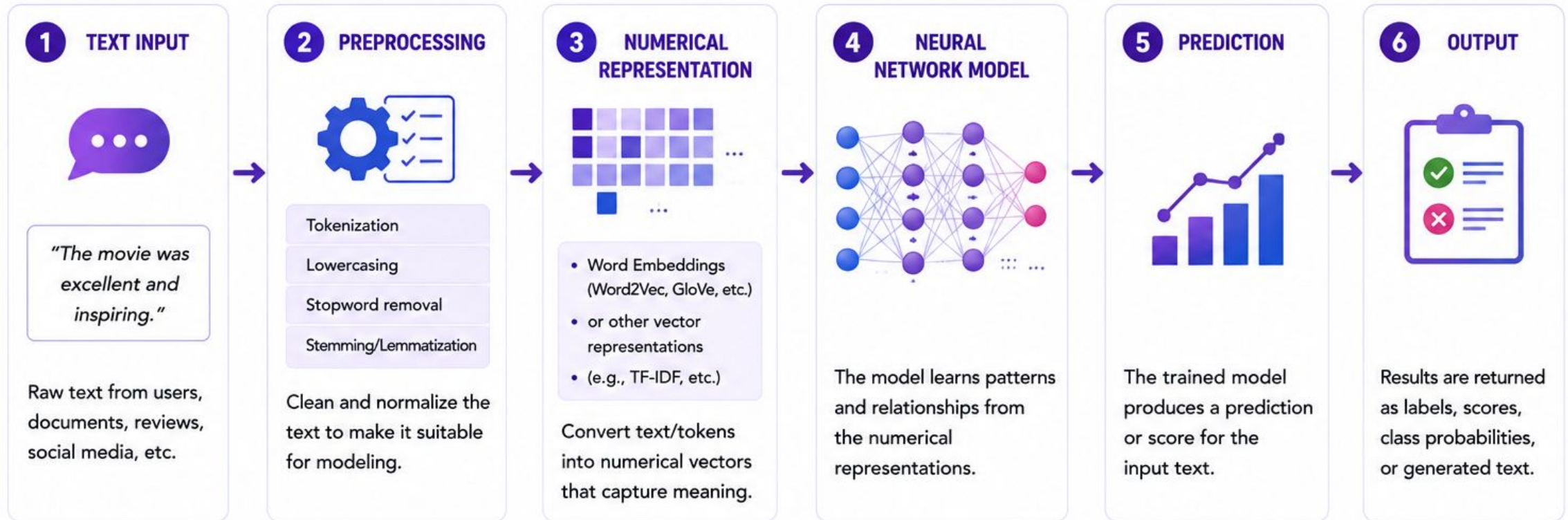


## KEY TAKEAWAY

Traditional ML relies on manual features and works well for simpler tasks, while neural networks learn representations automatically and excel at complex NLP problems.

# The NLP Neural Network Pipeline

An NLP system transforms raw text into meaningful predictions through a sequence of well-defined steps.



## KEY TAKEAWAY

The NLP pipeline converts raw text into predictions by preprocessing the text, representing it numerically, and using a neural network to learn and make predictions.



# What is an Artificial Neural Network?



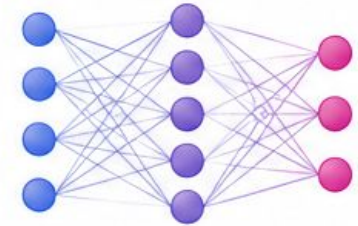
An Artificial Neural Network (ANN) is a computing model inspired by the **human brain**.

It is made up of many simple processing units (neurons) connected together to **learn patterns from data**.

Human Brain



Artificial Neural Network



## KEY CHARACTERISTICS



Composed of **simple units (neurons)** connected by **weighted links**.



Learns from data by **adjusting the weights** of the connections.



Uses **activation functions** to model **non-linear** relationships.

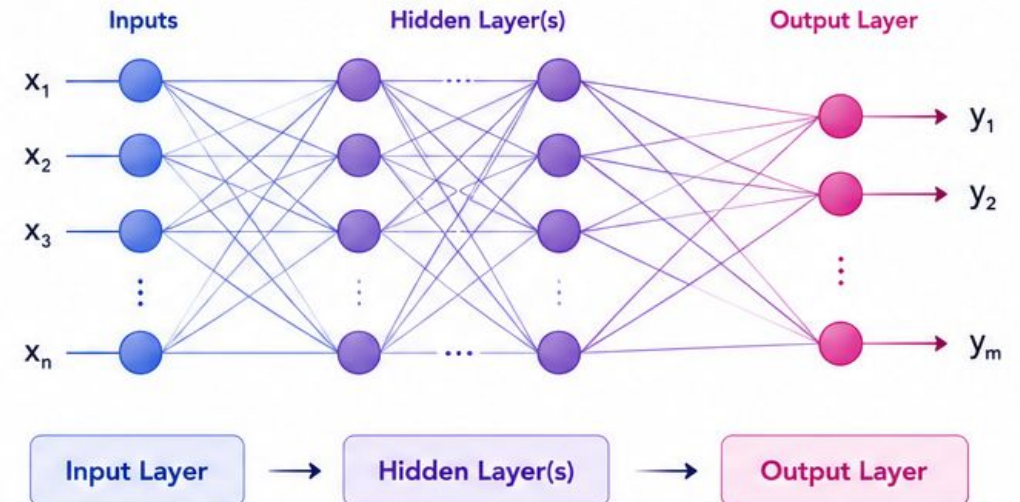


Organized in layers: **input layer**, one or more **hidden layers**, and **output layer**.



Trained to **minimize error** and make **accurate predictions**.

## Basic Structure of an ANN



## KEY TAKEAWAY

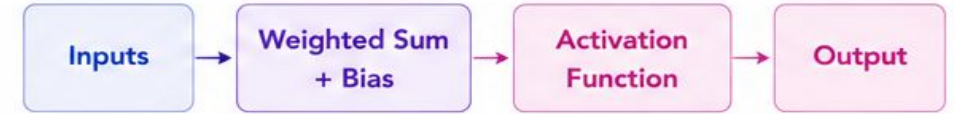
An Artificial Neural Network is a collection of connected neurons organized in layers. It **learns from data** by discovering patterns and making predictions.

# The Artificial Neuron



An artificial neuron is the **basic building block** of a neural network. It receives inputs, applies **weights**, adds a **bias**, passes the result through an **activation function**, and produces an **output**.

## How it Works (in a Neuron)



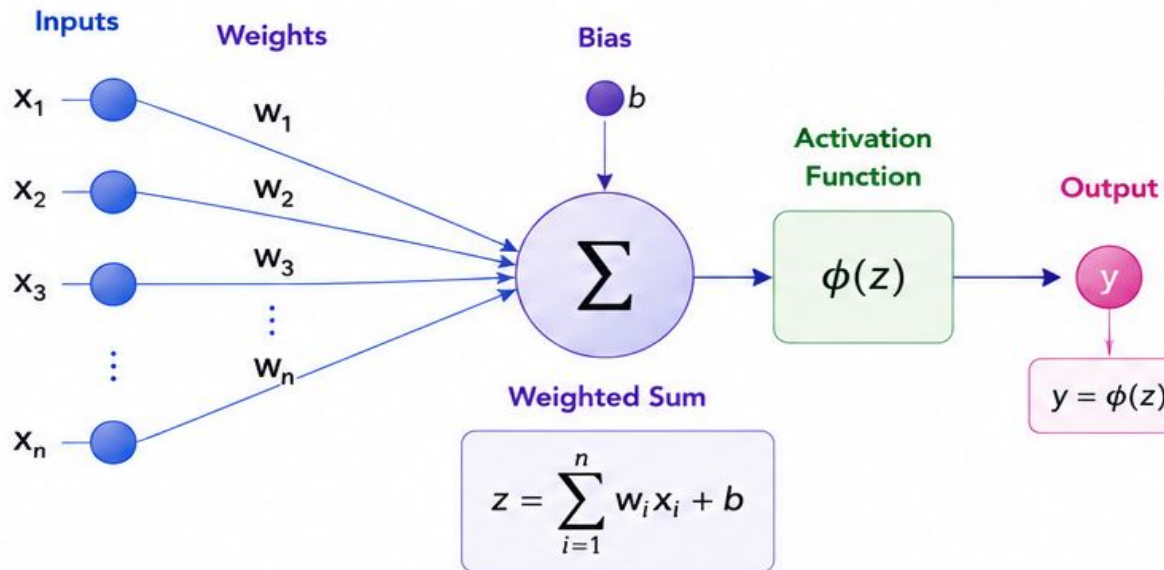
Multiple inputs come into the neuron.

Each input is multiplied by a weight, summed, and bias is added.

A non-linear function is applied to introduce complexity.

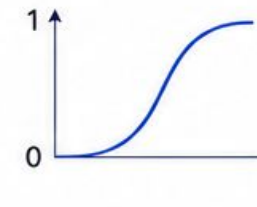
The neuron produces an output that is passed to the next layer.

## Structure of an Artificial Neuron



## Common Activation Functions

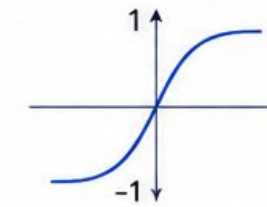
### Sigmoid



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Outputs between 0 and 1.

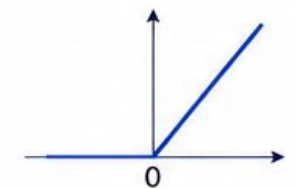
### tanh



$$\tanh(z)$$

Outputs between -1 and 1.

### ReLU



$$\text{ReLU}(z) = \max(0, z)$$

Outputs 0 for negative values; linear for positive values.



## KEY TAKEAWAY

An artificial neuron takes multiple **inputs**, applies **weights** and **bias**, uses an **activation function**, and produces an **output**.



# Inputs, Weights and Bias

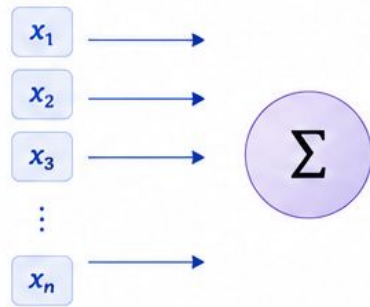
An artificial neuron receives inputs, multiplies them by weights, adds a bias, and passes the result through an activation function to produce an output.

## IN A NUTSHELL

- Inputs carry information.
- Weights control the importance of each input.
- Bias shifts the activation function and allows more flexibility.
- Together, they determine the output of a neuron.

### 1 INPUTS

Inputs are the values or features that enter a neuron.

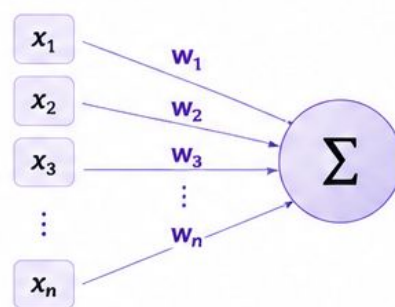


#### Examples

- Word embedding values
- TF-IDF features
- Numeric or categorical features

### 2 WEIGHTS

Weights determine the importance of each input.

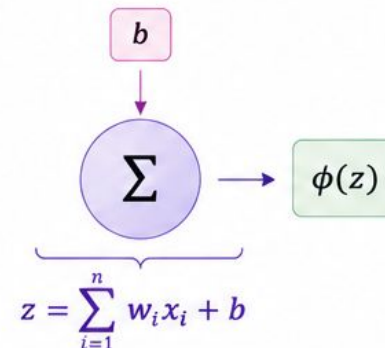


#### Examples

- Larger  $|w_i| \rightarrow$  more influence
- Positive weight  $\rightarrow$  increases output
- Negative weight  $\rightarrow$  decreases output

### 3 BIAS

Bias shifts the input to the activation function, allowing more flexibility.



#### Examples

- Acts like an adjustable threshold
- Lets the model fit data better
- Helps avoid outputs stuck at zero

### HOW THEY WORK TOGETHER

The neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b$$

Then applies an activation function:

$$y = \phi(z)$$

Where:

- $x_i$  = input  $i$
- $w_i$  = weight for input  $i$
- $b$  = bias term
- $\phi$  = activation function
- $y$  = output



## KEY TAKEAWAY

**Inputs** provide information, **weights** control their influence, and **bias** adjusts the threshold. Together, they determine the **output** of a neuron.

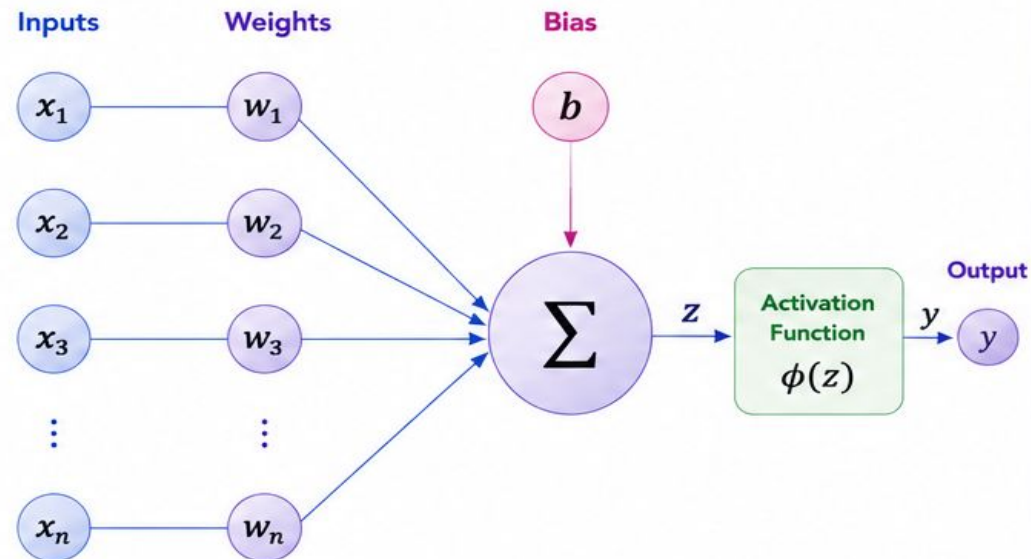
# How a Neuron Computes an Output

A neuron takes **inputs**, multiplies them by their **weights**, adds a **bias**, and passes the result through an **activation function** to produce an **output**.

## IN A NUTSHELL

- 1 Multiply each input by its weight.
- 2 Add all the products.
- 3 Add the bias.
- 4 Apply the activation function.
- 5 Get the output.

### A Single Neuron



### Computation Steps

#### 1 Weighted sum of inputs

$$z = \sum_{i=1}^n w_i x_i$$

Multiply each input  $x_i$  by its weight  $w_i$  and add them up.

#### 2 Add bias

$$z = z + b$$

Add the bias term  $b$ .

#### 3 Apply activation function

$$y = \phi(z)$$

Pass  $z$  through the activation function  $\phi$ .

#### 4 Output

$y$  is the output of the neuron.

### Example

Given:

$$\begin{aligned} x_1 &= 0.6, & w_1 &= 0.2 \\ x_2 &= -1.0, & w_2 &= -0.5 \\ x_3 &= 2.0, & w_3 &= 1.0 \\ b &= 0.1 \end{aligned}$$

Step-by-step:

$$\begin{aligned} \text{1 } z &= (0.6 \times 0.2) + (-1.0 \times -0.5) + (2.0 \times 1.0) \\ &= 0.12 + 0.50 + 2.00 = 2.62 \end{aligned}$$

$$\text{2 } z = 2.62 + 0.1 = 2.72$$

$$\begin{aligned} \text{3 } \text{Using sigmoid } \phi(z) &= \frac{1}{1 + e^{-z}} \\ y = \phi(2.72) &= \frac{1}{1 + e^{-2.72}} \approx 0.938 \end{aligned}$$

$$\text{4 Output: } y \approx 0.938$$

The neuron outputs  $y \approx 0.938$



## KEY TAKEAWAY

A neuron computes its output by taking a **weighted sum** of inputs, adding a **bias**, and applying an **activation function**.



# Weighted Sum – Worked Example

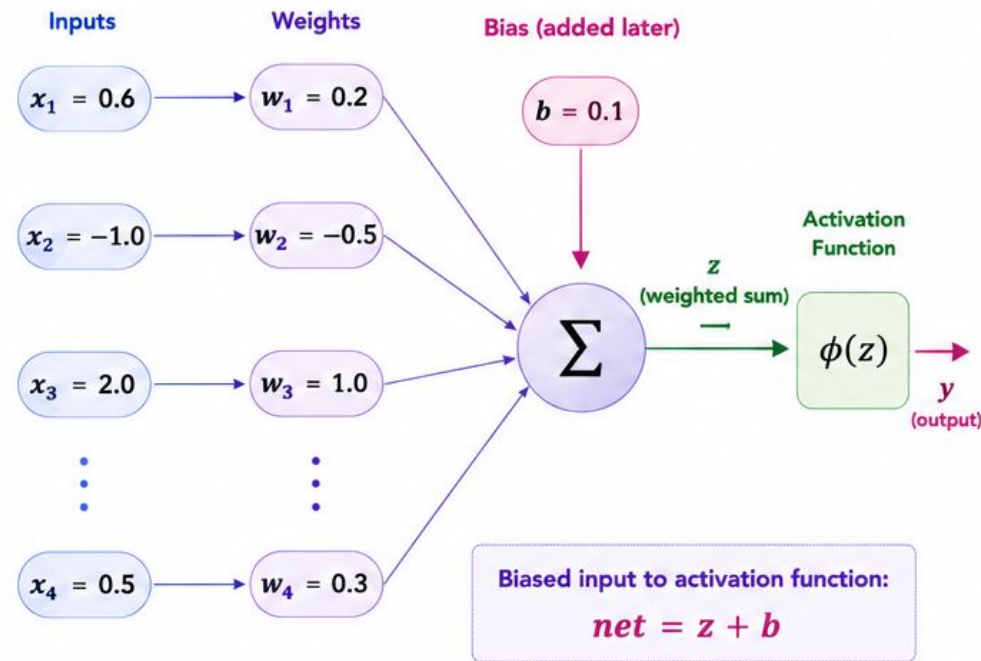
The weighted sum ( $z$ ) is the total input to the neuron **before the activation function** is applied. It is computed as the sum of each input multiplied by its weight.



## IMPORTANT

- Weighted sum does **NOT** include the bias.
- Bias is added after computing the weighted sum.
- This determines how strongly each input influences the neuron.

### 1. THE NEURON



### 2. FORMULA – WEIGHTED SUM

Weighted Sum ( $z$ ):

$$z = \sum_{i=1}^n w_i x_i$$

Where:

- $x_i$  = input  $i$
- $w_i$  = weight for input  $i$
- $n$  = number of inputs

### 3. WORKED EXAMPLE – CALCULATE THE WEIGHTED SUM ( $z$ )

| Input ( $x_i$ ) | Weight ( $w_i$ ) | $x_i \times w_i$              | Explanation                           |
|-----------------|------------------|-------------------------------|---------------------------------------|
| $x_1 = 0.6$     | $w_1 = 0.2$      | $0.6 \times 0.2 = 0.12$       | 0.6 times 0.2                         |
| $x_2 = -1.0$    | $w_2 = -0.5$     | $(-1.0) \times (-0.5) = 0.50$ | Negative $\times$ negative = positive |
| $x_3 = 2.0$     | $w_3 = 1.0$      | $2.0 \times 1.0 = 2.00$       | 2.0 times 1.0                         |
| $x_4 = 0.5$     | $w_4 = 0.3$      | $0.5 \times 0.3 = 0.15$       | 0.5 times 0.3                         |

Weighted Sum  $z = \sum w_i x_i = 0.12 + 0.50 + 2.00 + 0.15 = 2.77$

### 4. NEXT STEP – ADD THE BIAS AND APPLY ACTIVATION FUNCTION

Add bias:

$$net = z + b = 2.77 + 0.1 = 2.87$$

Apply activation function:

$$y = \phi(net) = \phi(2.87)$$



## KEY TAKEAWAY

- The **weighted sum** is the sum of each input multiplied by its weight.
- It measures the total evidence **before** bias and activation are considered.
- In this example, the weighted sum  $z = 2.77$ .

# What is an Activation Function?

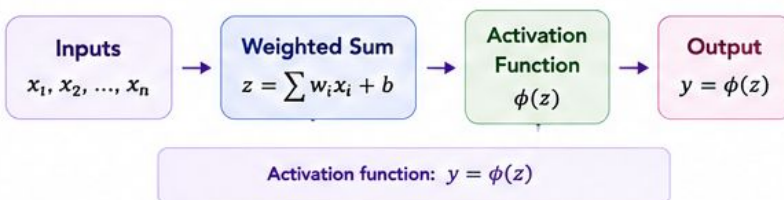


An **activation function** is a mathematical function applied to a neuron's **weighted sum (plus bias)** to produce the **output**. It introduces **non-linearity**, allowing the neural network to learn complex patterns in data.

## KEY POINTS

- Without activation functions, a neural network would be just a **linear model**.
- They help the network learn complex, **non-linear relationships**.
- Different activation functions are used in **hidden layers** and **output layers**.
- They are **differentiable** so gradients can be computed for learning.

## HOW IT WORKS



## WHY DO WE NEED IT?

- ✓ Adds **non-linearity** to the model.
- ✓ Allows the network to learn **complex functions and patterns**.
- ✓ Enables **decision boundaries** that are not just straight lines.

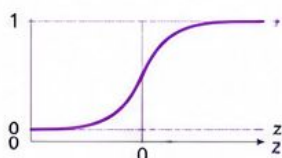
## WHERE ARE THEY USED?

- Hidden Layers:** ReLU, tanh, Leaky ReLU, ELU, etc.
- Output Layer:** Sigmoid (binary), Softmax (multi-class), Linear (regression)

## COMMON ACTIVATION FUNCTIONS

### Sigmoid (Logistic)

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

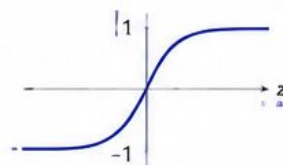


Range:  $(0, 1)$

Used in binary classification (output layer).

### tanh (Hyperbolic Tangent)

$$\phi(z) = \tanh(z)$$

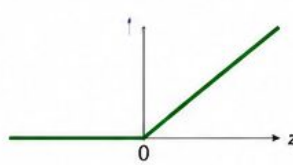


Range:  $(-1, 1)$

Zero-centered; used in hidden layers.

### ReLU (Rectified Linear Unit)

$$\phi(z) = \max(0, z)$$



Range:  $[0, \infty)$

Most popular in hidden layers; computationally efficient.

### Leaky ReLU

$$\phi(z) = \max(\alpha z, z)$$

( $\alpha$  small, e.g., 0.01)

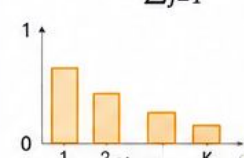


Range:  $(-\infty, \infty)$

Allows small gradient when  $z < 0$ ; avoids "dying ReLU" problem.

### Softmax (for multi-class)

$$\phi(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

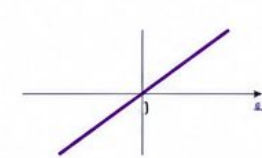


Range:  $(0, 1)$

Outputs probabilities that sum to 1.

### Linear (Identity)

$$\phi(z) = z$$



Range:  $(-\infty, \infty)$

Used for regression (output layer).



## KEY TAKEAWAY

Activation functions transform the weighted sum into meaningful outputs and enable neural networks to model complex, **non-linear relationships**.

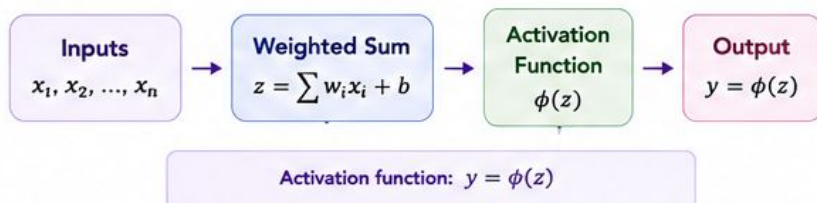


# What is an Activation Function?



An **activation function** is a mathematical function applied to a neuron's **weighted sum (plus bias)** to produce the **output**. It introduces **non-linearity**, allowing the neural network to learn complex patterns in data.

## HOW IT WORKS



## WHY DO WE NEED IT?

- ✓ Adds non-linearity to the model.
- ✓ Allows the network to learn complex functions and patterns.
- ✓ Enables decision boundaries that are not just straight lines.

## WHERE ARE THEY USED?

- Hidden Layers:** ReLU, tanh, Leaky ReLU, ELU, etc.
- Output Layer:** Sigmoid (binary), Softmax (multi-class), Linear (regression)

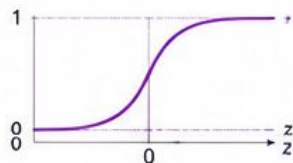
## KEY POINTS

- Without activation functions, a neural network would be just a **linear model**.
- They help the network learn complex, **non-linear relationships**.
- Different activation functions are used in **hidden layers** and **output layers**.
- They are **differentiable** so gradients can be computed for learning.

## COMMON ACTIVATION FUNCTIONS

### Sigmoid (Logistic)

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

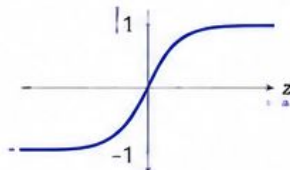


Range: (0, 1)

Used in binary classification (output layer).

### tanh (Hyperbolic Tangent)

$$\phi(z) = \tanh(z)$$

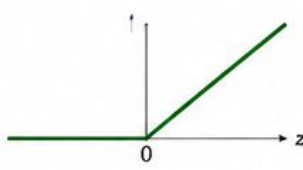


Range: (-1, 1)

Zero-centered; used in hidden layers.

### ReLU (Rectified Linear Unit)

$$\phi(z) = \max(0, z)$$



Range: [0, ∞)

Most popular in hidden layers; computationally efficient.

### Leaky ReLU

$$\phi(z) = \max(\alpha z, z)$$

( $\alpha$  small, e.g., 0.01)

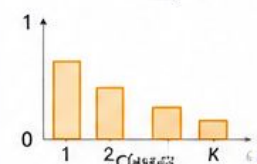


Range: (-∞, ∞)

Allows small gradient when  $z < 0$ ; avoids "dying ReLU" problem.

### Softmax (for multi-class)

$$\phi(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

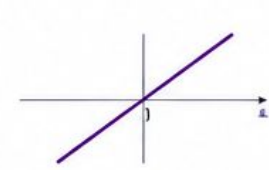


Range: (0, 1)

Outputs probabilities that sum to 1.

### Linear (Identity)

$$\phi(z) = z$$



Range: (-∞, ∞)

Used for regression (output layer).



## KEY TAKEAWAY

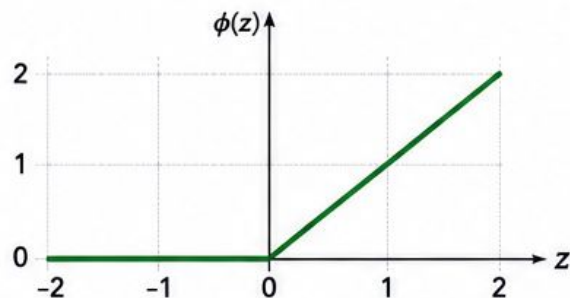
Activation functions transform the weighted sum into meaningful outputs and enable neural networks to model complex, **non-linear relationships**.

# ReLU (Rectified Linear Unit)



**ReLU** is one of the most commonly used activation functions in neural networks. It introduces **non-linearity** while keeping the computation **simple** and **efficient**. It outputs **zero** for negative inputs and the **input itself** for positive inputs.

## GRAPH



- For  $z \leq 0$ , output is 0 (flat line).
- For  $z > 0$ , output increases linearly with slope 1.

## EXAMPLE

| Input (z) | ReLU(z) = max(0, z) | Explanation                   |
|-----------|---------------------|-------------------------------|
| -3        | 0                   | Negative → output is 0        |
| -1        | 0                   | Negative → output is 0        |
| -0.5      | 0                   | Negative → output is 0        |
| 0         | 0                   | Zero → output is 0            |
| 0.5       | 0.5                 | Positive → output is the same |
| 2         | 2                   | Positive → output is the same |
| 3.5       | 3.5                 | Positive → output is the same |

## DERIVATIVE

The derivative of ReLU is:

$$\phi'(z) = \begin{cases} 0, & \text{if } z < 0 \\ 1, & \text{if } z > 0 \\ \text{undefined (or 0 or 1),} & \text{if } z = 0 \end{cases}$$

This simple gradient makes backpropagation **computationally efficient**.



In practice, the value at  $z = 0$  is set to 0 or 1; it does not affect learning.

## WHY REUL IS POPULAR?

- ✓ Computationally simple and fast.
- ✓ Helps mitigate the **vanishing gradient** problem.
- ✓ Encourages **sparse** activations (many zeros), which can improve generalization.
- ✓ Works very well in practice for deep neural networks including NLP models.

## WHERE IS ReLU USED?



Hidden layers of deep neural networks (CNNs, RNNs, LSTMs, Transformers, etc.)



Common in NLP models such as text classifiers, sequence models, and feed-forward networks.

## KEY TAKEAWAY



ReLU keeps things simple:  
**no complex math, just  $\max(0, z)$ !**

It is a key reason deep neural networks train faster and perform better.



# Tanh (Hyperbolic Tangent)



The tanh (hyperbolic tangent) function is a **non-linear** activation function that maps any real-valued input to the **range**  $(-1, 1)$ . It is **zero-centered**, meaning outputs are balanced around 0, which often leads to faster convergence during training.

## DEFINITION

The tanh function is defined as:

$$\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

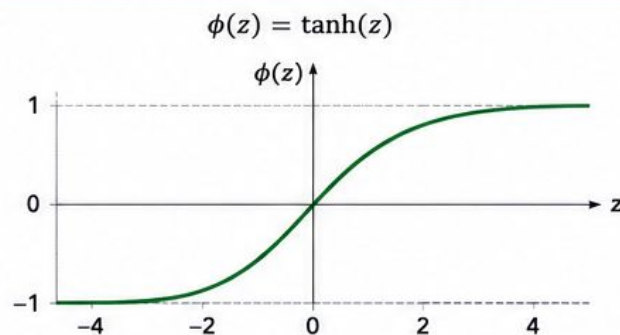
Where:

- $z$  = weighted sum of inputs (plus bias)
- $e$  = Euler's number  $\approx 2.71828$

## KEY PROPERTIES

- ✓ Outputs are between  $-1$  and  $1$ .
- ✓ Zero-centered: mean output  $\sim 0$ .
- ✓ Smooth and continuous.
- ✓ Monotonically increasing.
- ✓ Differentiable everywhere.
- ✓ Often used in hidden layers.

## GRAPH



- As  $z \rightarrow +\infty$ ,  $\tanh(z) \rightarrow 1$
- As  $z \rightarrow -\infty$ ,  $\tanh(z) \rightarrow -1$
- At  $z = 0$ ,  $\tanh(0) = 0$

## EXAMPLE VALUES

| $z$  | $\tanh(z)$ | Interpretation    |
|------|------------|-------------------|
| -3   | -0.9951    | Close to -1       |
| -2   | -0.9640    | Negative (strong) |
| -1   | -0.7616    | Negative          |
| -0.5 | -0.4621    | Negative (small)  |
| 0    | 0.0000     | Neutral (zero)    |
| 0.5  | 0.4621     | Positive (small)  |
| 1    | 0.7616     | Positive          |
| 2    | 0.9640     | Positive (strong) |
| 3    | 0.9951     | Close to 1        |

## DERIVATIVE

The derivative of tanh is:

$$\phi'(z) = 1 - \tanh^2(z)$$



Since  $-1 \leq \tanh(z) \leq 1$ , we have  $0 \leq \phi'(z) \leq 1$ .

The derivative is maximum at  $z = 0$ :

$$\phi'(0) = 1$$

and approaches 0 as  $z \rightarrow \pm\infty$ .

## WHERE IS TANH USED?



Hidden layers in many neural networks (especially older architectures).



Models where zero-centered activations help with learning.

## COMPARISON: TANH vs SIGMOID vs ReLU

| Function       | Range         | Zero-Centered? | Output Shape       | Common Use            |
|----------------|---------------|----------------|--------------------|-----------------------|
| <b>Sigmoid</b> | $(0, 1)$      | No             | S-shaped (0 to 1)  | Output layer (binary) |
| <b>Tanh</b>    | $(-1, 1)$     | Yes            | S-shaped (-1 to 1) | Hidden layers         |
| <b>ReLU</b>    | $[0, \infty)$ | No             | Piecewise linear   | Hidden layers         |

## KEY TAKEAWAY



Tanh squashes inputs to  $(-1, 1)$  and is zero-centered, which can lead to faster and more stable learning compared to sigmoid.



## REMEMBER

Tanh is similar to the sigmoid function but outputs range from  $-1$  to  $1$  instead of  $0$  to  $1$ . Its zero-centered nature makes it a great choice for hidden layers in many neural network models.



# Softmax

## (Multi-Class Activation Function)



Softmax is an activation function used in the output layer of **multi-class classification** models.

It converts a vector of real-valued scores (logits) into a **probability distribution** over  $K$  classes.

All probabilities are between **0 and 1** and **sum to 1**.

### DEFINITION

Given a vector of logits (scores)  $z = [z_1, z_2, \dots, z_K]$ , the softmax function computes:

$$\phi(z_i) = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Where:

- $z_i$  = logit (unnormalized score) for class  $i$
- $K$  = total number of classes
- $e$  = Euler's number  $\approx 2.71828$



Softmax exponentiates each score and normalizes by the sum of exponentials to produce probabilities.

### KEY PROPERTIES

- ✓ Outputs are in the range (0, 1).
- ✓ The probabilities sum to 1.
- ✓ Emphasizes larger scores and suppresses smaller ones.
- ✓ Continuously differentiable.
- ✓ Used in the output layer for multi-class classification.
- ✓ Interpretation:  $\phi(z_i)$  is the model's estimated probability for class  $i$ .

### INTUITION

Softmax turns scores into probabilities.



Higher scores  $\rightarrow$  higher probabilities.

All probabilities add up to 1 (100%).

### WORKED EXAMPLE

Suppose a model outputs the following logits for  $K = 4$  classes:

| Class   | Logit ( $z_i$ ) |
|---------|-----------------|
| Class 1 | 2.0             |
| Class 2 | 1.0             |
| Class 3 | 0.1             |
| Class 4 | -1.0            |

Step 1: Exponentiate each logit

| Class | $z_i$ | $e^{z_i}$ |
|-------|-------|-----------|
| 1     | 2.0   | 7.3891    |
| 2     | 1.0   | 2.7183    |
| 3     | 0.1   | 1.1052    |
| 4     | -1.0  | 0.3679    |

Sum of exponentials:  $\sum_{j=1}^4 e^{z_j} = 7.3891 + 2.7183 + 1.1052 + 0.3679 = 11.5805$

Step 2: Compute softmax

| Class | $e^{z_i}$ | Softmax $\phi(z_i)$         |
|-------|-----------|-----------------------------|
| 1     | 7.3891    | $7.3891 / 11.5805 = 0.6381$ |
| 2     | 2.7183    | $2.7183 / 11.5805 = 0.2349$ |
| 3     | 1.1052    | $1.1052 / 11.5805 = 0.0954$ |
| 4     | 0.3679    | $0.3679 / 11.5805 = 0.0317$ |

Output probabilities = [0.6381, 0.2349, 0.0954, 0.0317] (Sum = 1.0000 ✓)

### DERIVATIVE (JACOBIAN)

The partial derivative is:

$$\frac{\partial \phi_i}{\partial z_j} = \phi_i (\delta_{ij} - \phi_j)$$

Where  $\delta_{ij}$  is the Kronecker delta:

$$\delta_{ij} = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases}$$



This makes softmax differentiable and suitable for backpropagation.

### NUMERICAL STABILITY

To avoid overflow from large logits, subtract the maximum logit before exponentiating:

$$\phi(z_i) = \frac{e^{(z_i - z_{\max})}}{\sum_{j=1}^K e^{(z_j - z_{\max})}}$$



This does not change the output probabilities, but improves stability.

### WHERE IS SOFTMAX USED?



Output layer of multi-class classification models (e.g., sentiment analysis with 3 classes, POS tagging with many tags).



Sequence-to-sequence models (e.g., machine translation) to predict the next word from a vocabulary.

### SOFTMAX vs SIGMOID

| Aspect   | Softmax                      | Sigmoid                     |
|----------|------------------------------|-----------------------------|
| Use case | Multi-class ( $K > 2$ )      | Binary ( $K = 2$ )          |
| Output   | $K$ probabilities (sum to 1) | Single probability (0 or 1) |
| Range    | (0, 1) for each class        | (0, 1)                      |
| Sum      | Sum = 1                      | No constraint               |
| Example  | Classify into 5 topics       | Spam vs Not Spam            |



### KEY TAKEAWAY

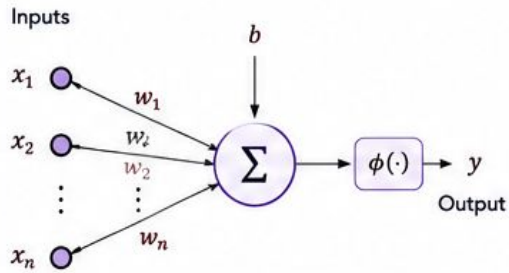
Softmax converts any set of scores into a valid probability distribution over classes. It is the standard choice for **multi-class classification** in neural networks.



# From One Neuron to a Neural Network

A neural network is built by connecting **many neurons** together in **layers**. These connections allow the network to learn **complex patterns** from data.

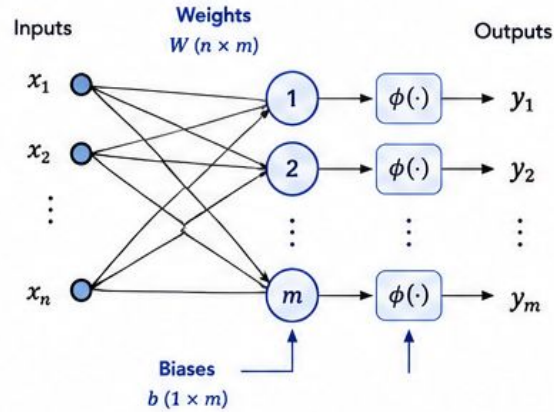
## 1. ONE NEURON



### Computation

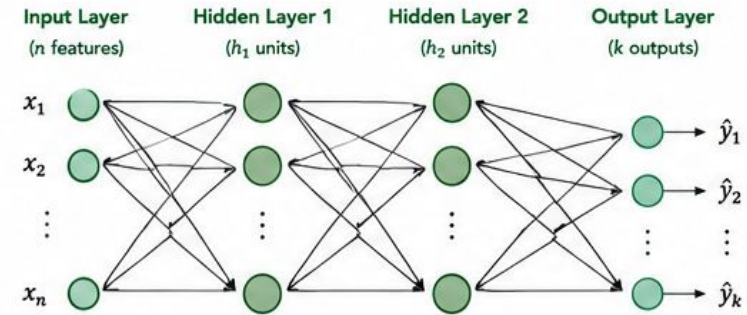
$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$
$$y = \phi(z)$$

## 2. LAYER OF NEURONS



Each neuron receives all inputs, applies weights and bias, then an activation function to produce its output.

## 3. MULTI-LAYER NEURAL NETWORK

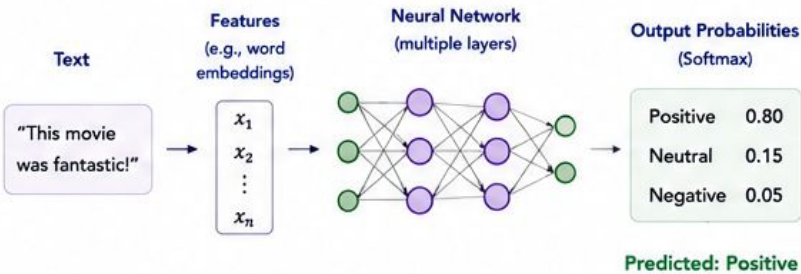


Information flows forward through layers. The network learns the best weights and biases to minimize prediction error.

## SUMMARY OF THE FLOW

- 1 Each connection has a weight.
- 2 Each neuron computes a weighted sum, adds a bias, then applies an activation function.
- 3 Layers of neurons transform the data step-by-step.
- 4 The final layer produces the prediction.

## EXAMPLE: CLASSIFYING TEXT (SENTIMENT)



## KEY TAKEAWAY



One neuron is the basic building block. Many neurons in layers form a neural network capable of learning complex relationships in data—such as understanding and generating human language.



From a single neuron to deep neural networks, the core idea remains the same: learn the right transformations of data to make accurate predictions.

More layers and neurons → more expressive power, but also more data and computation.

# Input, Hidden and Output Layers



A neural network is organized into **layers of neurons**.

The three main types are: **Input Layer**, **Hidden Layer(s)** and **Output Layer**.

Information flows from left to right during the forward pass.

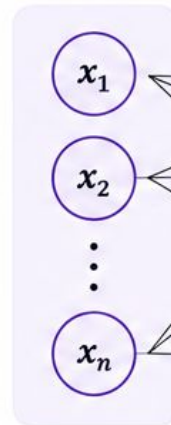
## INFORMATION FLOW

**Inputs** → **Hidden Layer(s)** → **Output**

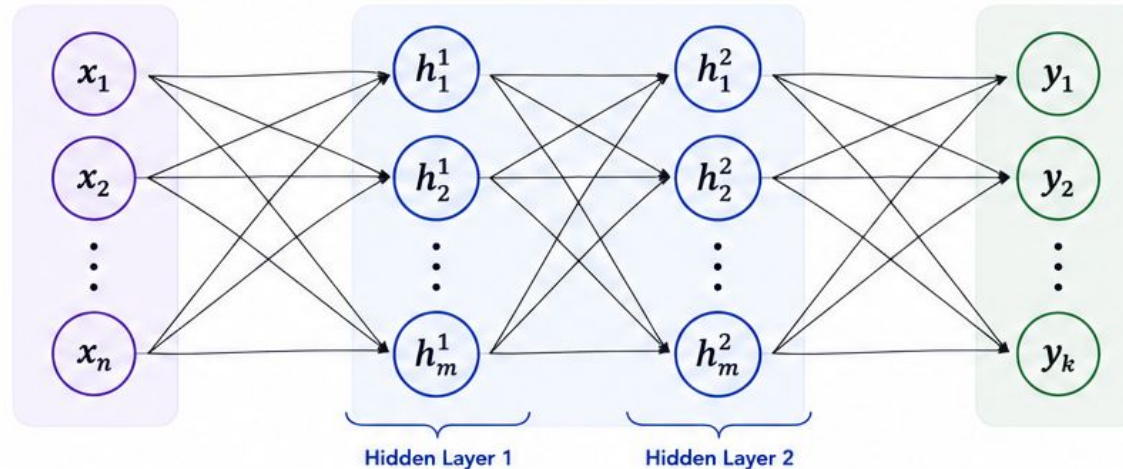
The **input layer** receives the **raw data**.

- Each node represents a **feature**.
- It does **not** perform any computations.
- The data is simply **passed** to the next layer.

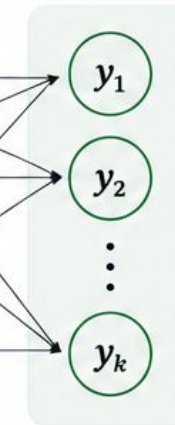
### INPUT LAYER



### HIDDEN LAYER(S)



### OUTPUT LAYER



The **output layer** produces the **final prediction**.

- Each node corresponds to an **output**.
- The activation function depends on the task (e.g., **softmax** for classification).
- Gives the network's **answer**.

#### 1. INPUT LAYER



- Size = number of input features ( $n$ )
- **Example** (text classification): Word embeddings, TF-IDF values, token IDs, etc.

**Example:** A sentence is converted into a list of numbers and fed to the input layer.

#### 2. HIDDEN LAYER(S)



- One or more hidden layers.
- Each neuron learns a representation (feature) from the previous layer.
- Uses activation functions like ReLU, tanh, etc.

**Example:** The network learns patterns such as words, phrases, or sentence meaning.

#### 3. OUTPUT LAYER



- Size = number of outputs ( $k$ )
- For classification:  $k$  = number of classes
- For regression:  $k$  = 1 (or more values)

**Example:** For sentiment analysis (3 classes), outputs might be: [0.05, 0.15, 0.80]



## KEY TAKEAWAY

➤ **Input layer** = receives data

➤ **Hidden layer(s)** = learn and extract useful patterns

➤ **Output layer** = produces the final result

Together, these layers enable the neural network to learn complex relationships from data.



# What Happens in the Hidden Layer?



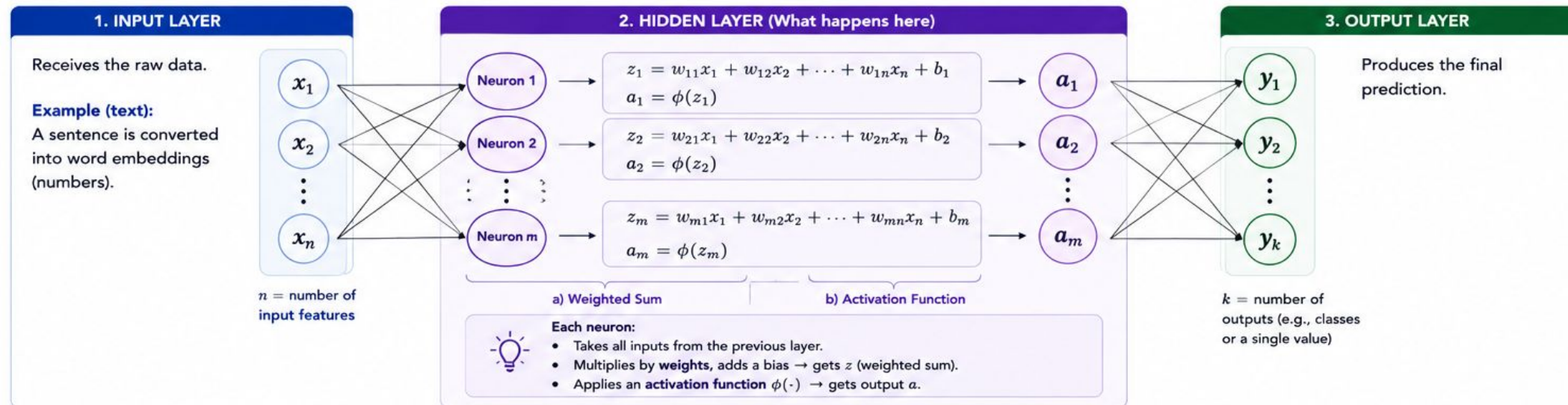
The hidden layer is where the neural network does most of its “thinking”.

It **transforms** the input into more **useful representations** that help the model make accurate predictions.

## BIG PICTURE

Input → Hidden Layer(s) → Output

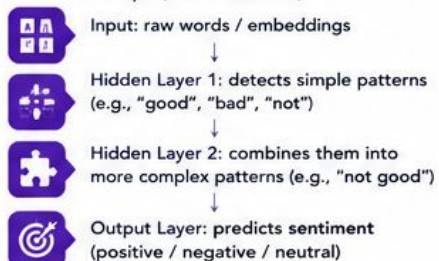
Hidden layers take simple inputs and convert them into higher-level features that are easier to solve.



## WHAT THE HIDDEN LAYER LEARNS

It learns representations (features) that capture patterns in the data.

Example (text classification):



## COMMON ACTIVATION FUNCTIONS

**ReLU:**  $\phi(z) = \max(0, z)$



**Tanh:**  $\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

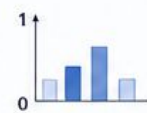


**Sigmoid:**  $\phi(z) = \frac{1}{1 + e^{-z}}$



**Softmax** (output layer):

$$\phi_i(z) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$



## WHY HIDDEN LAYERS MATTER

- ✓ Capture non-linear relationships.
- ✓ Build hierarchical features (from simple → complex).
- ✓ Allow the network to generalize to new data.
- ✓ More hidden layers/neurons → more expressive power (but needs more data and computational resources).

## KEY TAKEAWAY



The hidden layer is the engine of the neural network.  
It converts raw inputs into meaningful features through weighted sums and activation functions.  
These learned features enable the model to solve complex tasks.



## IN SHORT

In the hidden layer, each neuron combines inputs with learned weights and bias, applies an activation function, and passes the result forward. This creates powerful features that help the network make accurate predictions.



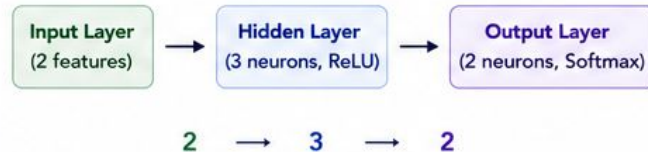
# Forward Propagation — Worked Example



Forward propagation computes the output of a neural network given input, weights and biases.

We pass the input forward through each layer using:  $z = Wa + b$  and  $a = \phi(z)$

## NETWORK ARCHITECTURE



### 1. GIVEN

Input vector (column)

$$a^{[0]} = x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

Hidden layer (2 → 3)

$$W^{[1]} = \begin{bmatrix} 0.5 & -0.3 \\ -0.2 & 0.8 \\ 0.1 & 0.4 \end{bmatrix}$$

$$b^{[1]} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.0 \end{bmatrix}$$

Output layer (3 → 2)

$$W^{[2]} = \begin{bmatrix} 0.7 & -0.6 & 0.2 \\ -0.1 & 0.4 & 0.5 \end{bmatrix}$$

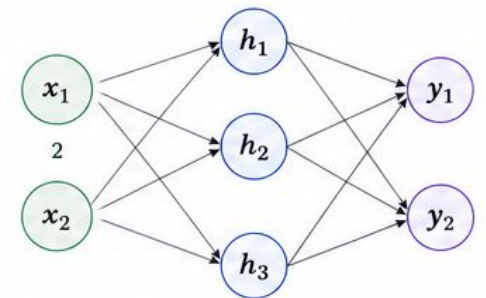
$$b^{[2]} = \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix}$$

Activation functions:

Hidden: ReLU  $\phi(z) = \max(0, z)$

Output: Softmax  $\phi(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

### 2. NETWORK DIAGRAM



Input Layer  
 $a^{[0]} (2 \times 1)$

Hidden Layer (ReLU)  
 $a^{[1]} (3 \times 1)$

Output Layer (Softmax)  
 $a^{[2]} (2 \times 1)$

### 3. STEP-BY-STEP COMPUTATION

#### STEP 1: HIDDEN LAYER

$$\begin{aligned} z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\ &= \begin{bmatrix} 0.5 & -0.3 \\ -0.2 & 0.8 \\ 0.1 & 0.4 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.0 \end{bmatrix} \\ &= \begin{bmatrix} 0.5(2) + (-0.3)(-1) \\ -0.2(2) + 0.8(-1) \\ 0.1(2) + 0.4(-1) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.0 \end{bmatrix} \\ &= \begin{bmatrix} 1.0 + 0.3 \\ -0.4 - 0.8 \\ 0.2 - 0.4 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 1.3 \\ -1.2 \\ -0.2 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} a^{[1]} &= \text{ReLU}(z^{[1]}) = \max(0, z^{[1]}) \\ &= \begin{bmatrix} \max(0, 1.3) \\ \max(0, -1.2) \\ \max(0, -0.2) \end{bmatrix} = \begin{bmatrix} 1.3 \\ 0 \\ 0 \end{bmatrix} \end{aligned}$$

#### STEP 2: OUTPUT LAYER (LOGITS)

$$\begin{aligned} z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\ &= \begin{bmatrix} 0.7 & -0.6 & 0.2 \\ -0.1 & 0.4 & 0.5 \end{bmatrix} \begin{bmatrix} 1.3 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix} \\ &= \begin{bmatrix} 0.7(1.3) + (-0.6)(0) + 0.2(0) \\ -0.1(1.3) + 0.4(0) + 0.5(0) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix} \\ &= \begin{bmatrix} 0.91 \\ -0.13 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix} = \begin{bmatrix} 1.01 \\ -0.43 \end{bmatrix} \end{aligned}$$

#### STEP 3: OUTPUT LAYER (SOFTMAX)

$$\begin{aligned} a^{[2]} &= \text{softmax}(z^{[2]}) \\ &= \frac{1}{e^{1.01} + e^{-0.43}} \begin{bmatrix} e^{1.01} \\ e^{-0.43} \end{bmatrix} \\ &\approx \frac{1}{2.745} \begin{bmatrix} 2.745 \\ 0.651 \end{bmatrix} = \begin{bmatrix} 1.000 \\ 0.237 \end{bmatrix} \approx \begin{bmatrix} 0.763 \\ 0.237 \end{bmatrix} \end{aligned}$$

### 4. SUMMARY OF FORWARD PROPAGATION

| Layer                | Computation                           | Result                                                     |
|----------------------|---------------------------------------|------------------------------------------------------------|
| Input ( $a^{[0]}$ )  | Given input                           | $\begin{bmatrix} 2 \\ -1 \end{bmatrix}$                    |
| Hidden ( $z^{[1]}$ ) | $z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$ | $\begin{bmatrix} 1.3 \\ -1.2 \\ -0.2 \end{bmatrix}$ $-0.2$ |
| Hidden ( $a^{[1]}$ ) | $a^{[1]} = \text{ReLU}(z^{[1]})$      | $\begin{bmatrix} 1.3 \\ 0 \\ 0 \end{bmatrix}$ $0$          |
| Output ( $z^{[2]}$ ) | $z^{[2]} = W^{[2]} a^{[1]} + b^{[2]}$ | $\begin{bmatrix} 1.01 \\ -0.43 \end{bmatrix}$              |
| Output ( $a^{[2]}$ ) | $a^{[2]} = \text{softmax}(z^{[2]})$   | $\begin{bmatrix} 0.763 \\ 0.237 \end{bmatrix}$             |

### 5. INTERPRETATION

- Neuron  $h_1$  is activated (1.3), while  $h_2$  and  $h_3$  are deactivated (0) by ReLU.
- The network uses these features to compute the final scores for  $y_1$  and  $y_2$ .
- After softmax, the output is a probability distribution over the 2 classes.

**Prediction: Class 1 with 76.3% probability**

### 6. KEY TAKEAWAYS

- ✓ Forward propagation moves data forward only.
- ✓ At each layer: weighted sum → add bias → activation → pass to next layer.
- ✓ Hidden layers learn useful representations (features).
- ✓ The output layer produces the final prediction (probabilities for classification).

### FORMULAS AT A GLANCE

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = \phi(z^{[l]})$$

$$\text{ReLU: } \phi(z) = \max(0, z)$$

$$\text{Softmax: } \phi(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



**FINAL OUTPUT (PREDICTION):**

$$a^{[2]} = \begin{bmatrix} 0.763 \\ 0.237 \end{bmatrix} \Rightarrow \text{Class 1 (76.3% probability), Class 2 (23.7% probability)}$$



# How Does the Network Make a Prediction?



A neural network makes a prediction by **transforming the input step-by-step** through layers and producing an output that represents the answer.

## THE BIG PICTURE



## THE 5 STEPS: FROM INPUT TO PREDICTION

### 1. RECEIVE INPUT

Provide the features.

Example (text classification):

The sentence  
"This movie is fantastic!"  
is converted into a numerical vector.

Input vector  $x$

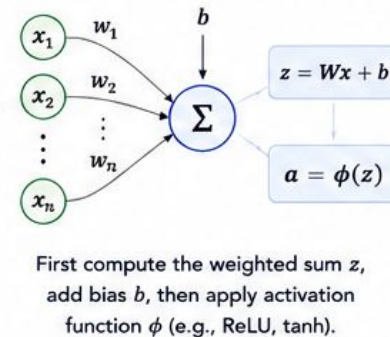
$$\begin{bmatrix} 0.12 \\ 0.85 \\ 0.33 \\ \vdots \\ 0.05 \end{bmatrix} \quad (n \times 1)$$


Raw data → numbers

### 2. COMPUTE HIDDEN LAYERS

Extract useful patterns.

For each neuron in a hidden layer:

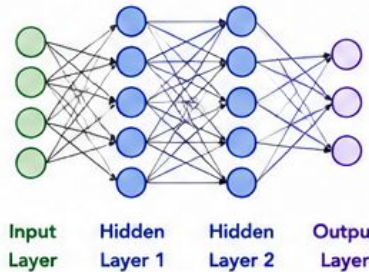


Learned weights + activation create features

### 3. REPEAT THROUGH LAYERS

Build higher-level features.

The output of one layer becomes the input to the next.



Network learns increasingly complex representations

### 4. COMPUTE OUTPUT LAYER

Produce final scores (logits).

The last layer outputs raw scores (logits) for each possible class/value.

$$z^{(L)} = W^{(L)}a^{(L-1)} + b^{(L)}$$

Example (3 classes):

$$z^{(L)} = \begin{bmatrix} 2.10 \\ 0.30 \\ -1.70 \end{bmatrix}$$



Scores indicate how well each class matches

### 5. CONVERT TO PREDICTION

Turn scores into decision.

Apply activation for output:

- Softmax (classification)
- Linear (regression)

Softmax example: Probabilities:

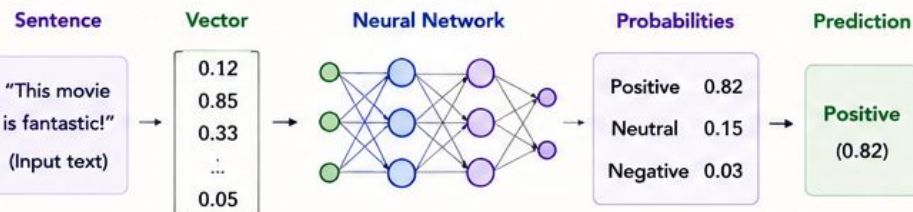
$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = \begin{bmatrix} 0.82 \\ 0.15 \\ 0.03 \end{bmatrix}$$

Prediction:  
**Class 1 (0.82)**



Choose class with highest probability (argmax)

## EXAMPLE: SENTIMENT CLASSIFICATION



## WHAT'S HAPPENING?

- 1 Input numbers go into the network.
- 2 Hidden layers transform the input using learned weights and activation functions.
- 3 The output layer produces scores for each class.
- 4 Scores are converted to probabilities.
- 5 The class with the highest probability is the prediction.

## KEY TAKEAWAY



A neural network makes a prediction by passing input forward through layers, applying weights, biases and activation functions, and converting the final output into the most likely answer.



**IN SHORT:** Input → Hidden layers (transform & extract patterns) → Output layer (scores) → Probabilities → Prediction (final answer)

This forward pass happens in milliseconds and is used during both training (to compute loss) and inference (to make predictions).



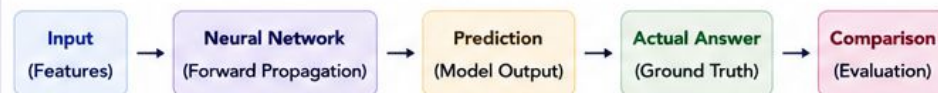
# Prediction vs Actual Answer



A neural network outputs a **prediction**.

We compare this prediction with the **actual (ground truth) answer** to evaluate how well the model performed.

## THE FLOW: FROM INPUT TO COMPARISON



We compare the **prediction** with the **actual answer** and measure the error or correctness.

### EXAMPLE: TEXT CLASSIFICATION (3 CLASSES)

| No. | Input Text                    | Actual Answer (Ground Truth) | Model Prediction (Probabilities)  | Predicted Class (argmax) | Correct? (Yes / No) |
|-----|-------------------------------|------------------------------|-----------------------------------|--------------------------|---------------------|
| 1   | This movie is fantastic!      | Positive                     | [0.82, 0.15, 0.03]<br>Pos Neu Neg | Positive                 | ✓ Yes               |
| 2   | The food was okay.            | Neutral                      | [0.20, 0.60, 0.20]<br>Pos Neu Neg | Neutral                  | ✓ Yes               |
| 3   | I did not like the service.   | Negative                     | [0.25, 0.30, 0.45]<br>Pos Neu Neg | Negative                 | ✓ Yes               |
| 4   | It was a waste of time.       | Negative                     | [0.40, 0.35, 0.25]<br>Pos Neu Neg | Positive                 | ✗ No                |
| 5   | It was okay, nothing special. | Neutral                      | [0.33, 0.40, 0.27]<br>Pos Neu Neg | Neutral                  | ✓ Yes               |

Pos = Positive (Class 1)

Neu = Neutral (Class 2)

Neg = Negative (Class 3)

### HOW WE COMPARE

#### Classification (Softmax Output)

- Model outputs probabilities for each class.  
Example: [0.82, 0.15, 0.03]
- Predicted class = class with highest probability (argmax).  
Here: Class 1 (Positive)
- Compare predicted class with actual class.
  - If same → **Correct**
  - If different → **Incorrect**

#### Regression (Linear Output)

- Model outputs a numeric value.  
Example:  $\hat{y} = 3.72$
- Compare with actual value  $y = 4.20$
- Compute error (e.g., MAE, MSE):
  - Absolute Error =  $|y - \hat{y}| = 0.48$
  - Squared Error =  $(y - \hat{y})^2 = 0.2304$



Goal: Minimize error (regression) or maximize accuracy (classification).

### CONFUSION MATRIX (FOR THE EXAMPLE ABOVE)

Rows = Actual | Columns = Predicted

| Actual \ Predicted | Positive (Class 1) | Neutral (Class 2) | Negative (Class 3) | Total |
|--------------------|--------------------|-------------------|--------------------|-------|
| Positive (Class 1) | 1                  | 0                 | 0                  | 1     |
| Neutral (Class 2)  | 0                  | 2                 | 0                  | 2     |
| Negative (Class 3) | 1                  | 0                 | 1                  | 2     |
| Total              | 2                  | 2                 | 1                  | 5     |



Correct Predictions



Incorrect Predictions

### EVALUATION METRICS (FOR THIS EXAMPLE)

#### Accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

$$= \frac{4}{5} = 0.80 \text{ (80\%)}$$

#### Per-Class (Precision / Recall / F1)

| Class    | Precision | Recall | F1-Score |
|----------|-----------|--------|----------|
| Positive | 0.50      | 1.00   | 0.67     |
| Neutral  | 1.00      | 1.00   | 1.00     |
| Negative | 1.00      | 0.50   | 0.67     |

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

### KEY TAKEAWAY



- The model's **prediction** is compared with the **actual (ground truth) answer**.
- This comparison tells us how well the model is performing.
- We use **metrics** to measure performance and guide learning.

### CLASS LABELS (IN THIS EXAMPLE)

Class 1: **Positive**

Class 2: **Neutral**

Class 3: **Negative**



**IN SHORT:** Prediction = what the model outputs

Actual Answer = the correct (ground truth) label or value

Compare → Measure → Improve



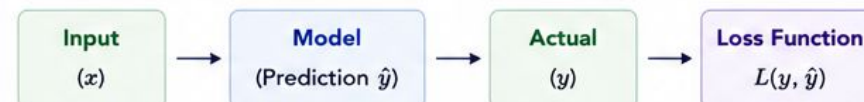
# What is a Loss Function?



A **loss function** (or cost function) measures **how far** the model's prediction is from the actual (true) answer.

It gives a **single number** that tells us how bad (or good) the model is doing.

## THE BIG PICTURE: WHERE LOSS IS USED



Loss tells us how wrong the prediction is.  
During training, we try to **minimize** this loss.

## 1. WHY DO WE NEED A LOSS FUNCTION?

- ✓ The model makes predictions.
- ✓ We need a way to measure the error.
- ✓ The loss function converts that error into a number.
- ✓ We use this number to update the model's weights and biases.



**Goal of training:**  
Minimize the loss.

## 2. COMMON LOSS FUNCTIONS

### A. Classification (e.g., Softmax Output)

#### Cross-Entropy Loss

Used for multi-class classification.  
Measures the difference between the true class and predicted probabilities.

For one example:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where:

$y_i = 1$  if class  $i$  is the true class, else 0

$\hat{y}_i$  = predicted probability for class  $i$

$C$  = number of classes

### B. Regression (Linear Output)

#### Mean Squared Error (MSE)

Used for regression problems.  
Measures the average of squared differences.

For one example:

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

$y_i$  = actual value

$\hat{y}_i$  = predicted value

$n$  = number of outputs (often 1)

## 3. EXAMPLE CALCULATIONS

### Classification Example

3 classes. True class = Class 2

One-hot label  $y = [0, 1, 0]$

Predicted probabilities  $\hat{y} = [0.1, 0.7, 0.2]$

#### Cross-Entropy Loss:

$$\begin{aligned} L &= -(0 \log 0.1 + 1 \log 0.7 + 0 \log 0.2) \\ &= -\log 0.7 \\ &= 0.357 \end{aligned}$$

Lower loss → better prediction.  
(Perfect prediction would give loss = 0)

### Regression Example

Actual value  $y = 4.0$

Predicted value  $\hat{y} = 3.2$

#### MSE Loss:

$$\begin{aligned} L &= (4.0 - 3.2)^2 \\ &= (0.8)^2 \\ &= 0.64 \end{aligned}$$

Lower loss → prediction closer to the true value.

## 4. HOW LOSS DRIVES LEARNING



**Forward pass:** Model makes a prediction  $\hat{y}$ .



**Compute loss:** Compare  $\hat{y}$  with actual  $y$  using a loss function  $L(y, \hat{y})$ .

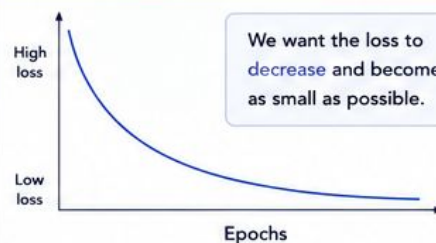


**Backward pass:** Use the loss to compute gradients.



**Update weights:** Adjust weights and biases to reduce the loss.

## 5. LOSS CURVE DURING TRAINING



If loss is high → model is far from the answer.  
If loss is low → model is doing well.

## 6. GOOD TO KNOW

- ✓ Loss is always  $\geq 0$ .
- ✓ Different tasks need different loss functions.
- ✓ We minimize loss during training, but we don't care about loss during inference (making predictions).
- ✓ A low loss usually means better predictions, but not always (check accuracy too).

## 7. KEY TAKEAWAY



A loss function tells us how wrong the model is.

It turns the error into a number so the model can learn.

Our goal in training:

**Minimize** the loss, improve the model.



## IN SHORT:

Prediction  $\hat{y}$  vs Actual  $y$  → Compute Loss  $L(y, \hat{y})$  → Use Loss to Improve the Model → Better Predictions!

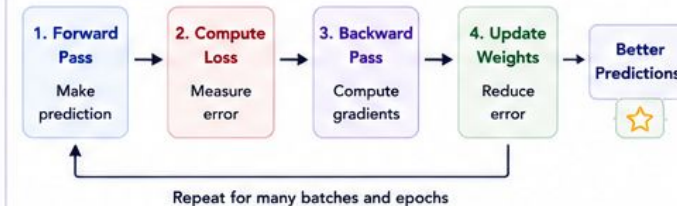


# How Does the Network Learn from Errors?



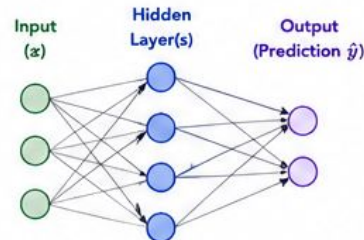
A neural network learns by comparing its **prediction** with the **actual answer**. It measures the **error (loss)** and **adjusts** its weights and biases to reduce that error. This process is repeated many times until the model makes better predictions.

## THE LEARNING CYCLE (REPEATED MANY TIMES)



### 1. FORWARD PASS

Input goes through the network to produce a prediction.



Example (classification with 2 classes):  
Predicted probabilities  $\hat{y} = [0.20, 0.80]$   
(Class 1, Class 2)

### 2. COMPUTE LOSS

Compare prediction  $\hat{y}$  with the **actual answer**  $y$  using a loss function  $L(y, \hat{y})$ .

**Example:**

True class is Class 2  $\rightarrow y = [0, 1]$

**Cross-Entropy Loss:**

$$L = - \sum_{i=1}^C y_i \ln(\hat{y}_i)$$

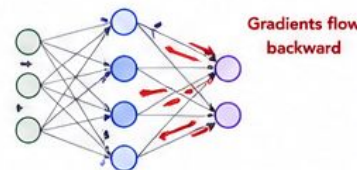
$$\begin{aligned} L &= -(0 \ln 0.20 + 1 \ln 0.80) \\ &= -\ln 0.80 \\ &= \mathbf{0.223} \end{aligned}$$

**Lower loss = better prediction**

### 3. BACKWARD PASS

Compute gradients: how much each weight and bias contributed to the error.

Using backpropagation + chain rule.



**What are gradients?**

They tell us the direction to change each weight/bias to reduce the loss.

### 4. UPDATE WEIGHTS

Adjust weights and biases in the direction that reduces the loss.

**Gradient Descent Update Rule**

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial L}{\partial b}$$

where  $\eta$  = learning rate  
(small step size)

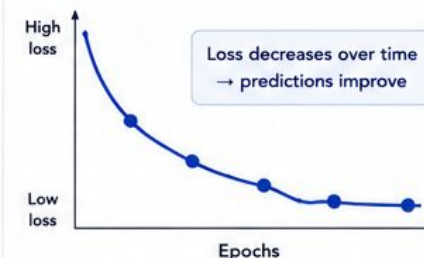
If gradient is **positive**  $\rightarrow$  decrease weight  
If gradient is **negative**  $\rightarrow$  increase weight



This update should reduce the loss.

### 5. REPEAT & IMPROVE

Repeat for many batches and epochs.



Eventually, the model converges (low loss) and makes good predictions on unseen data.



**Goal: Minimize the loss**

### NUMERICAL EXAMPLE (ONE WEIGHT)

| Step    | Weight $w$ | Gradient $\frac{\partial L}{\partial w}$ | Learning Rate $\eta$ | Update $-\eta \frac{\partial L}{\partial w}$ | New Weight $w_{\text{new}}$ | Loss $L$ |
|---------|------------|------------------------------------------|----------------------|----------------------------------------------|-----------------------------|----------|
| Initial | 0.50       | —                                        | 0.10                 | —                                            | 0.50                        | 0.223    |
| 1       | 0.50       | 0.40                                     | 0.10                 | -0.040                                       | 0.46                        | 0.195    |
| 2       | 0.46       | 0.30                                     | 0.10                 | -0.030                                       | 0.43                        | 0.170    |
| 3       | 0.43       | 0.20                                     | 0.10                 | -0.020                                       | 0.41                        | 0.148    |
| ...     | ...        | ...                                      | ...                  | ...                                          | ...                         | ...      |

Weights gradually change  $\rightarrow$  Loss goes down  $\rightarrow$  Predictions get better

### KEY TAKEAWAYS

- ✓ The network learns from its mistakes (errors).
- ✓ Loss measures how wrong the prediction is.
- ✓ Backpropagation tells us how each weight affects the loss.
- ✓ We update weights to reduce the loss.
- ✓ This process repeats again and again.
- ✓ Lower loss  $\rightarrow$  better model  $\rightarrow$  better predictions!

### COMMON LOSS FUNCTIONS

| Task Type                     | Common Loss Function         |
|-------------------------------|------------------------------|
| Classification (single-label) | Cross-Entropy Loss           |
| Classification (multi-label)  | Binary Cross-Entropy Loss    |
| Regression                    | Mean Squared Error (MSE)     |
| Ranking / Similarity          | Hinge Loss, Contrastive Loss |

### ANALOGY

Learning is like adjusting the aim of an archer.



After each shot (prediction), the archer sees how far off the target (error) is and adjusts the aim to get closer.



**IN SHORT:**



Make Prediction  
(Forward Pass)



Measure Error  
(Loss)



Find Cause of Error  
(Backpropagation)



Update Weights  
(Gradient Descent)



Lower Loss  
Better Predictions



# What is Backpropagation?



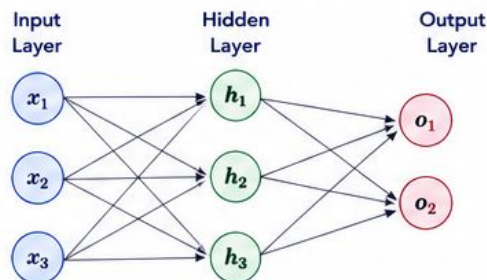
**Backpropagation** is the algorithm that tells the neural network how much each weight and bias **contributed to the error** (loss), so we can update them to make **better predictions**.

## THE BIG IDEA



We start from the output (where we see the error) and pass the error backward through the network to the earlier layers.

## 1. FORWARD PASS (We already did this)



Input (e.g., a sentence) → network computes output probabilities.

## 2. BACKWARD PASS (Backpropagation)

**Step 1:** Compute error at the output layer

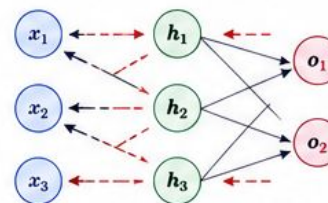
How wrong is our prediction? — (Compare with true answer)

**Step 2:** Send the error backward

Distribute the error to the hidden layer. Each connection gets a “blame” (gradient) for how much it caused the error.

**Step 3:** Continue backward

Repeat layer by layer until we reach the input layer. Now we know how much each weight and bias should change.



→ Forward pass (prediction)  
 ← Backward pass (error flow)

## HOW IT WORKS (Intuition)

- 1 The model makes a prediction.
- 2 We compute the loss (how wrong it is).
- 3 Backpropagation calculates gradients (derivatives of the loss) for every weight and bias.
- 4 Using these gradients, we update the weights so the loss becomes smaller.
- 5 We repeat this over many examples (epochs) until the model learns well.



**Backpropagation** does **NOT** change the architecture of the network. It only tells us how to adjust the numbers (weights and biases) inside the network.

## 3. SIMPLE NUMERICAL EXAMPLE (One weight)

Suppose one weight  $w$  connects  $h_2 \rightarrow o_1$

Current weight ( $w$ ) = 0.50  
 Prediction ( $\hat{y}_1$ ) = 0.80  
 True value ( $y_1$ ) = 1.00  
 Loss ( $L$ ) using MSE =  $(1.00 - 0.80)^2 = 0.04$

Gradient ( $\frac{\partial L}{\partial w}$ ) = -0.16 (computed by backprop)  
 Learning rate ( $\eta$ ) = 0.10

Update rule:  $w_{\text{new}} = w_{\text{old}} - \eta \times \frac{\partial L}{\partial w}$   
 $w_{\text{new}} = 0.50 - 0.10 \times (-0.16) = 0.516$   
 The weight increases slightly to reduce the loss.

## 4. WHAT ARE GRADIENTS?

- ✓ A gradient tells the direction and size of change needed for a weight/bias to reduce the loss.
- ✓ Positive gradient → decrease the weight.
- ✓ Negative gradient → increase the weight.
- ✓ Big gradient → large change.
- ✓ Small gradient → small change.

$\sum \frac{\partial L}{\partial w}$  Backpropagation uses the chain rule of calculus to compute gradients efficiently for all weights in the network.

## 5. SUMMARY

- ✓ Forward pass: compute output.
- ✓ Compute loss: how wrong is the prediction?
- ✓ Backpropagation: send error backward and compute gradients for all weights and biases.
- ✓ Update weights: move in the direction that reduces loss.
- ✓ Repeat for many batches and epochs.



Goal: minimize the loss → better predictions on new, unseen data!



## IN SHORT:



Forward Pass  
Make prediction



Compute Loss  
Measure error



Backpropagation  
Find gradients



Update Weights  
Reduce error



Repeat  
Improve model

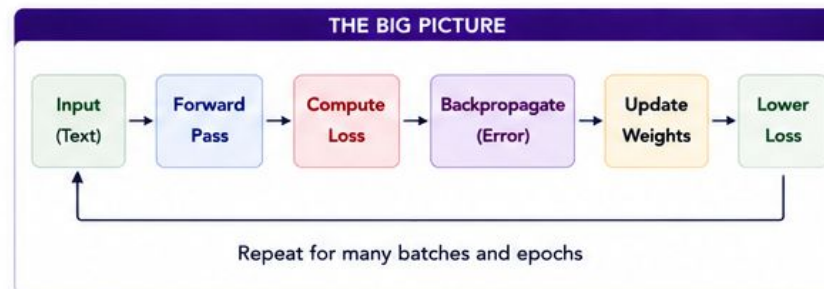


# Backpropagation — Conceptual Overview

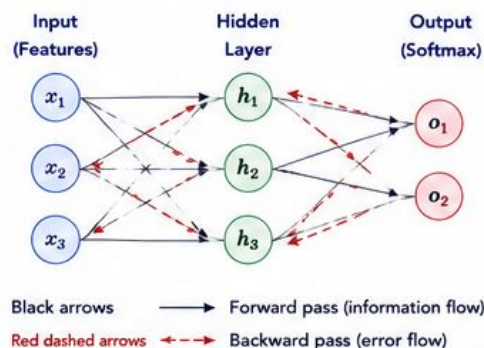


Backpropagation is the algorithm that **efficiently** computes how the error at the output is contributed by each weight and bias in the network.

This allows the model to update them and reduce the error.



## 1. THE NETWORK (Example)



## 2. FORWARD PASS (We already did this)

- Input text (e.g., "This movie is great!") is converted to features/tokens.
- The network computes activations layer by layer and produces output probabilities (e.g., sentiment).
- We get a prediction.

### Example:

Predicted probabilities ( $o$ ) = [0.10, 0.90]  
(True class: Positive → one-hot [0, 1])

## 3. COMPUTE LOSS

Compare prediction with the true answer using a loss function (e.g., cross-entropy).

### Example:

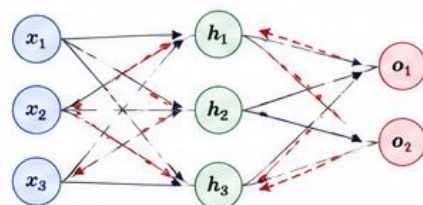
**Loss  $L = 0.357$**

This single number tells us how wrong the prediction is.

## 4. BACKWARD PASS (Backpropagation)

- Start at the output layer.**  
Compute how much each output neuron contributed to the loss.
- Propagate the error backward.**  
Use the chain rule to pass the error to the hidden layer.
- Continue layer by layer backward.**  
Until we reach the input layer.
- Now we know the **gradient** for every weight and bias in the network.  
(How much each one should change to reduce the loss.)

## VISUALIZING THE ERROR FLOW (Conceptual)



- Forward pass: left → right (black arrows) activations/probabilities flow forward.
- Backward pass: right → left (red dashed arrows) error signals flow backward.

Each connection gets a gradient  $\frac{\partial L}{\partial w}$  showing how it affects the loss.

## 5. UPDATE WEIGHTS

Use the gradients to update parameters.

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla w$$

$$b_{\text{new}} = b_{\text{old}} - \eta \nabla b$$

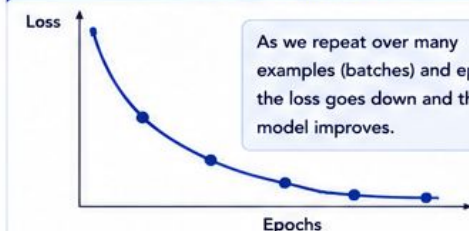
where:

$\eta$  = learning rate

$\nabla w, \nabla b$  = gradients from backpropagation

This moves the parameters in the direction that reduces the loss.

## 6. REPEAT & IMPROVE



## KEY TAKEAWAYS

- ✓ Backpropagation does not change the architecture.
- ✓ It just tells us how to change the numbers (weights and biases).
- ✓ It is based on calculus (chain rule) and is very efficient.
- ✓ It is the reason deep networks can be trained.

## WHAT ARE WE ACTUALLY COMPUTING?

| What                                                                        | Meaning                                | Computed in        |
|-----------------------------------------------------------------------------|----------------------------------------|--------------------|
| Loss ( $L$ )                                                                | How wrong is the prediction?           | After forward pass |
| Gradient ( $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$ ) | How each weight/bias affects the loss? | Backpropagation    |
| Updated weights                                                             | New values that reduce the loss        | After update step  |

## IN SHORT



**Goal:** Minimize the loss by using backpropagation to learn better weights and biases — leading to better predictions on new, unseen text.

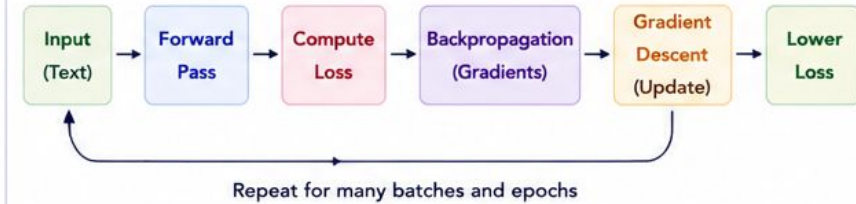


# Gradient Descent — Basic Concept



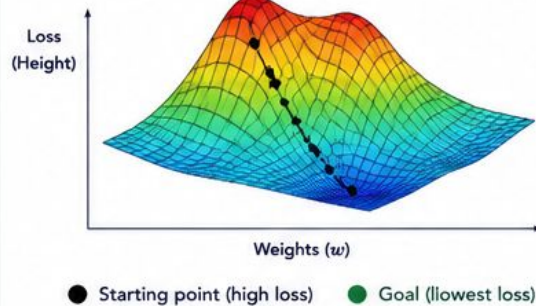
Gradient Descent is an optimization algorithm used to **minimize the loss**. It updates the model's weights and biases step by step in the direction that **reduces the loss the most**.

## WHERE GRADIENT DESCENT FITS IN



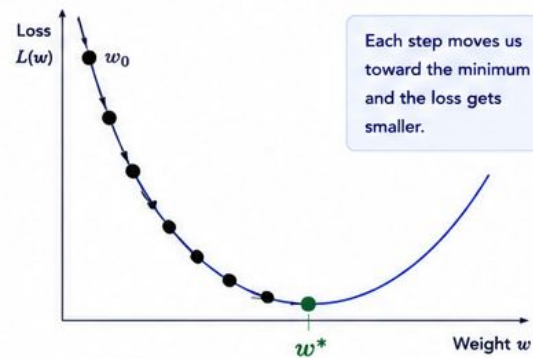
### 1. THE IDEA (Imagine a hilly landscape)

The loss is like the height of the landscape. Our goal is to go **DOWN** the hill to reach the lowest point (**minimum loss**).



### 2. EXAMPLE ON A SIMPLE LOSS CURVE

Loss as a function of one weight  $w$ . We start at  $w_0$  and try to reach the minimum.



### 4. NUMERICAL EXAMPLE (One weight)

Suppose the current weight  $w = 2.0$

Loss function:  $L(w) = (w - 1)^2$

Derivative (gradient):  $\nabla L(w) = 2(w - 1)$

Learning rate  $\eta = 0.1$

| Step      | $w$ (current) | Gradient $\nabla L(w)$ | Update $w_{new} = w - \eta \nabla L(w)$ | Loss $L(w)$ |
|-----------|---------------|------------------------|-----------------------------------------|-------------|
| 0 (start) | 2.000         | $2(2 - 1) = 2.000$     | $2.000 - 0.1 \times 2.000 = 1.800$      | 1.000       |
| 1         | 1.800         | $2(1.8 - 1) = 1.600$   | $1.800 - 0.1 \times 1.600 = 1.640$      | 0.640       |
| 2         | 1.640         | $2(1.64 - 1) = 1.280$  | $1.640 - 0.1 \times 1.280 = 1.512$      | 0.263       |
| 3         | 1.512         | $2(1.512 - 1) = 1.024$ | $1.512 - 0.1 \times 1.024 = 1.410$      | 0.169       |
| 4         | 1.410         | $2(1.410 - 1) = 0.820$ | $1.410 - 0.1 \times 0.820 = 1.328$      | 0.108       |
| ...       | ...           | ...                    | ...                                     | ...         |

The weight approaches  $w^* = 1.0$  and the loss approaches 0.

### 3. HOW DOES IT WORK?

At each step, we move a small distance in the direction of the negative gradient (steepest downhill).

Update rule:

$$w_{new} = w_{old} - \eta \nabla L(w)$$

where:

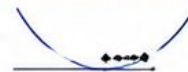
- $w$  = weights (or parameters)
- $\nabla L(w)$  = gradient of the loss with respect to  $w$
- $\eta$  = learning rate (step size)
- $-$  = move in the opposite direction of the gradient



Small steps lead us smoothly down to the valley.

### 5. EFFECT OF LEARNING RATE ( $\eta$ )

Too small ( $\eta$  very small)



Very slow convergence.

Good choice



Fast and stable.

Too large ( $\eta$  too big)



Overshoots or diverges.

### 6. KEY TAKEAWAYS

- ✓ Gradient Descent minimizes the loss by moving step by step.
- ✓ The gradient tells us the direction of the steepest increase.
- ✓ We move in the opposite direction (downhill).
- ✓ The learning rate controls the step size.
- ✓ Used in every neural network training!



Goal: Find weights that minimize the loss  
→ better predictions on new, unseen text!



## IN SHORT



Compute loss  
(How wrong?)



Compute gradient  
(Direction of steepest increase)



Update weights  
(Take a step downhill)



Repeat many times  
(Epochs)



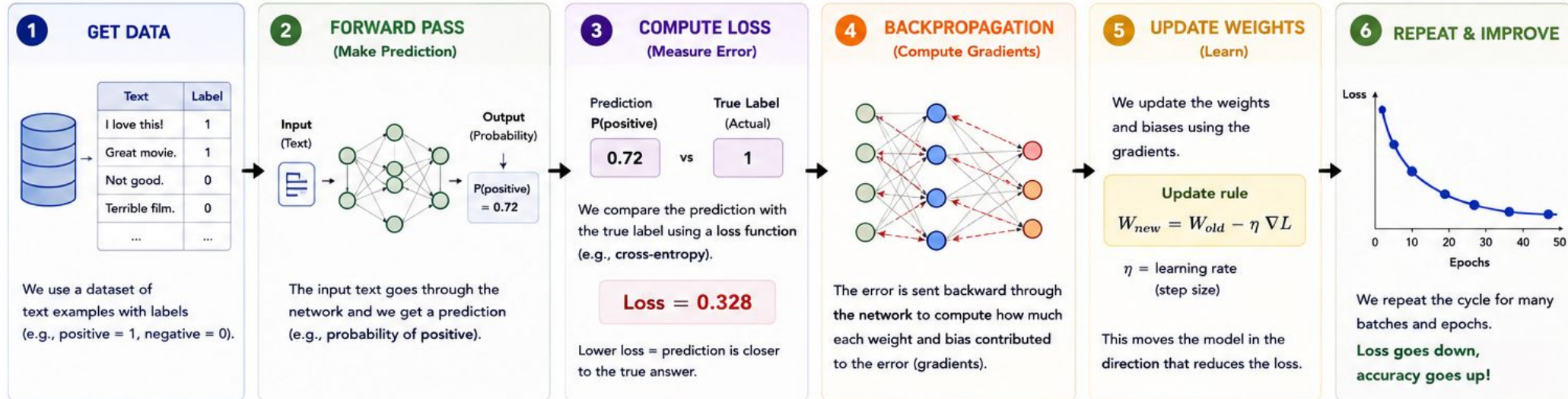
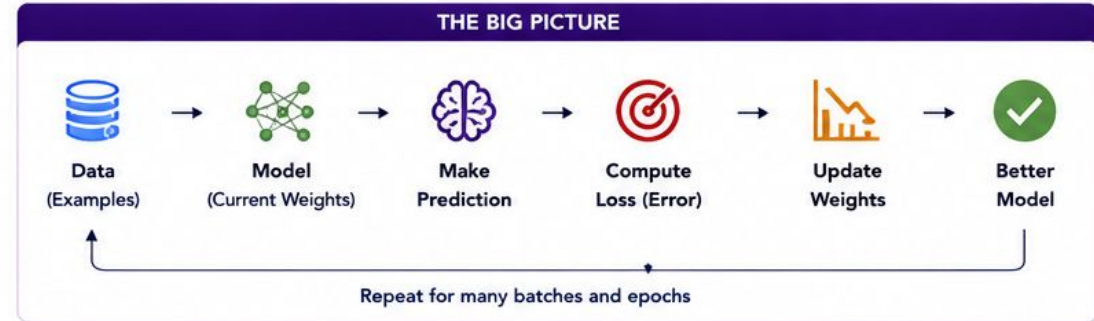
Lower loss  
Better model!



# The Training Cycle



Training a neural network is an **iterative** process. The model learns by making predictions, measuring errors, and updating itself to improve over time.



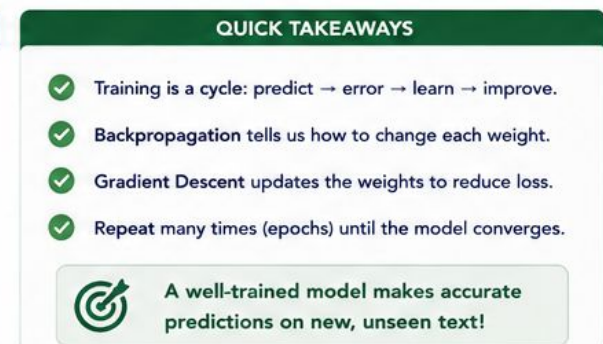
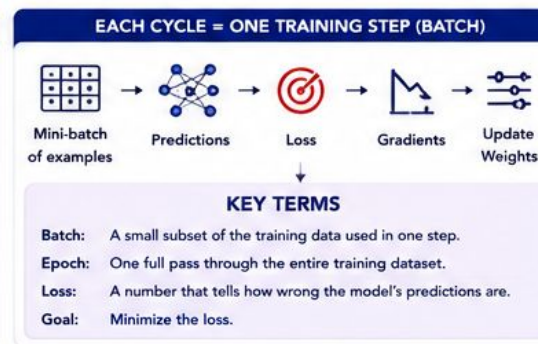
**EXAMPLE: SENTIMENT CLASSIFICATION**

| Text                    | True Label (Actual) | Prediction P(positive) | Loss (Cross-Entropy) |
|-------------------------|---------------------|------------------------|----------------------|
| I love this movie!      | 1 (Positive)        | 0.72                   | 0.328                |
| Great acting and story. | 1 (Positive)        | 0.81                   | 0.210                |
| Not good at all.        | 0 (Negative)        | 0.31                   | 0.371                |
| Terrible and boring.    | 0 (Negative)        | 0.12                   | 0.128                |

Average Loss (this batch) = **0.259**

The model predicts probabilities. We compute loss for each example, average it, and use it to update the model.

★ Over time, the model becomes better at predicting the right sentiment.



## IN SHORT



Get data



Forward pass (Predict)



Compute loss (Error)



Backpropagation (Gradients)



Update weights (Learn)



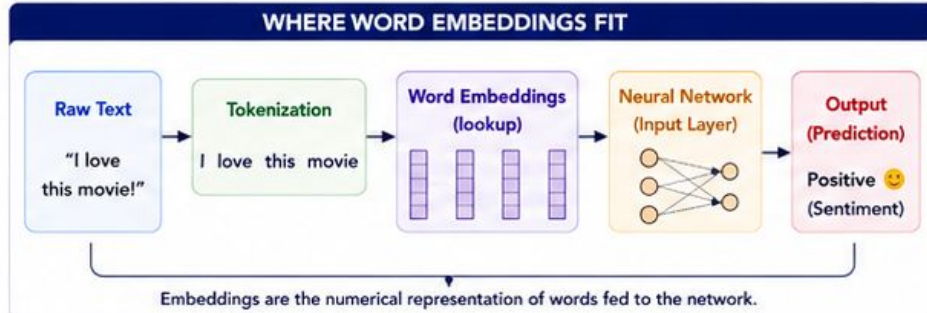
Repeat (Improve)



# Using Word Embeddings as Neural Network Inputs

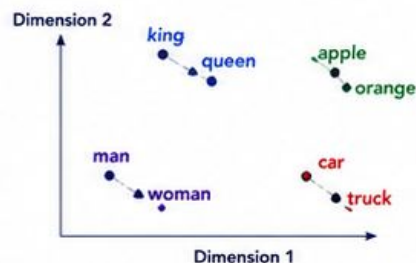


Word embeddings convert words into **dense numerical vectors** that capture meaning and context. These vectors are used as **inputs** to neural networks instead of sparse one-hot representations.



## 1. WHAT IS A WORD EMBEDDING?

- A word embedding maps each word to a low-dimensional dense vector of real numbers.
- Words with similar meanings have similar vectors.
- Embeddings are learned from large text corpora (e.g., Word2Vec, GloVe, FastText).
- Example: 4-dimensional embedding space.



Similar words are close together in the embedding space.

## 2. FROM TEXT TO EMBEDDING VECTORS

1. Tokenize the text into words.
2. Look up each word in the embedding matrix.
3. Use the vectors as input to the neural network.

Example Vocabulary (4D embeddings)

| Word  | $v_1$ | $v_2$ | $v_3$ | $v_4$ |
|-------|-------|-------|-------|-------|
| i     | 0.21  | -0.11 | 0.09  | 0.05  |
| love  | 0.31  | 0.23  | -0.10 | 0.17  |
| this  | 0.05  | -0.20 | 0.26  | -0.13 |
| movie | -0.12 | 0.30  | -0.05 | 0.22  |

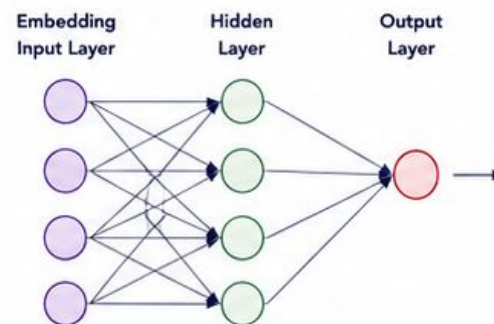
Embedding Matrix  $E \in \mathbb{R}^{V \times 4}$

|       | $v_1$    | $v_2$    | $v_3$    | $v_4$    |
|-------|----------|----------|----------|----------|
| i     | 0.21     | -0.11    | 0.09     | 0.05     |
| love  | 0.31     | 0.23     | -0.10    | 0.17     |
| this  | 0.05     | -0.20    | 0.26     | -0.13    |
| movie | -0.12    | 0.30     | -0.05    | 0.22     |
| ...   | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |

For the sentence: "I love this movie" → we get 4 vectors (one per word).

## 3. USING EMBEDDINGS AS NEURAL NETWORK INPUTS

### Example Network



For each word vector (size = 4), the network learns patterns and produces an output.

Output could be:

- Sentiment (positive/negative)
- Topic (news/sports/tech)
- Intent (question/command)
- Next word prediction, etc.

Input to network (for this sentence) =

|                                                            |                                                            |                                                             |                                                             |
|------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------|
| $\begin{bmatrix} 0.21 & -0.11 & 0.09 & 0.05 \end{bmatrix}$ | $\begin{bmatrix} 0.31 & 0.23 & -0.10 & 0.17 \end{bmatrix}$ | $\begin{bmatrix} 0.05 & -0.20 & 0.26 & -0.13 \end{bmatrix}$ | $\begin{bmatrix} -0.12 & 0.30 & -0.05 & 0.22 \end{bmatrix}$ |
| i                                                          | love                                                       | this                                                        | movie                                                       |

## 4. EXAMPLE: SENTIMENT CLASSIFICATION

Input Sentence

"This movie was fantastic"

Look up embeddings

| Word      | $e_1$ | $e_2$ | $e_3$ |
|-----------|-------|-------|-------|
| this      | 0.12  | -0.31 | 0.18  |
| movie     | -0.22 | 0.45  | -0.05 |
| was       | 0.08  | -0.12 | 0.32  |
| fantastic | 0.35  | 0.40  | 0.21  |

Embedding input matrix  $X \in \mathbb{R}^{4 \times 3}$

|           | $e_1$ | $e_2$ | $e_3$ |
|-----------|-------|-------|-------|
| this      | 0.12  | -0.31 | 0.18  |
| movie     | -0.22 | 0.45  | -0.05 |
| was       | 0.08  | -0.12 | 0.32  |
| fantastic | 0.35  | 0.40  | 0.21  |

Tokens:

this movie was fantastic

Assume we use 3D embeddings.

This matrix  $X$  is fed into the neural network to predict: **Positive** 😊

## 5. WHY USE WORD EMBEDDINGS?



### Compact Representation

Dense vectors are small and efficient (e.g., 100–300 dimensions).



### Capture Semantic Meaning

Similar words have similar vectors, helping the model generalize.



### Better than One-Hot

One-hot vectors are huge and sparse; embeddings are dense and informative.



### Learned from Data

They are trained on large text corpora so they understand context.

## KEY TAKEAWAYS

- ✓ Word embeddings convert words into dense numerical vectors.
- ✓ These vectors are used as inputs to neural networks.
- ✓ They capture semantic similarity and context.
- ✓ They are more efficient and powerful than one-hot encodings.
- ✓ They enable neural networks to learn better representations and make more accurate predictions.



Goal: Use word embeddings as inputs so the neural network can learn from meaning, not just word IDs!



# Worked Example – Sentiment Classification



**Goal:** Given a sentence, predict whether the sentiment is **Positive** or **Negative**

**Approach:** Use word embeddings → average → neural network → probability → class.

**Example sentence:** "I love this movie!"

## THE PIPELINE (OVERVIEW)



1. Text Input  
Raw sentence



2. Embeddings  
Word vectors



3. Neural Network  
Compute score



4. Probability  
Sigmoid output



5. Prediction  
Positive / Negative

### 1. TEXT INPUT

"I love this movie!"

After tokenization and lowercasing:

i love this movie !

(We will ignore the punctuation "!")

### 2. WORD EMBEDDINGS LOOKUP (Example 4D embeddings)

| Word  | $d_1$ | $d_2$ | $d_3$ | $d_4$ |
|-------|-------|-------|-------|-------|
| i     | 0.21  | -0.11 | 0.09  | 0.05  |
| love  | 0.31  | 0.23  | -0.10 | 0.17  |
| this  | 0.05  | -0.20 | 0.26  | -0.13 |
| movie | -0.12 | 0.30  | -0.05 | 0.22  |



These vectors come from a pre-trained embedding model (e.g., GloVe, Word2Vec, FastText).

### 3. AVERAGE TO GET SENTENCE VECTOR

Average pooling  
(mean of word vectors)

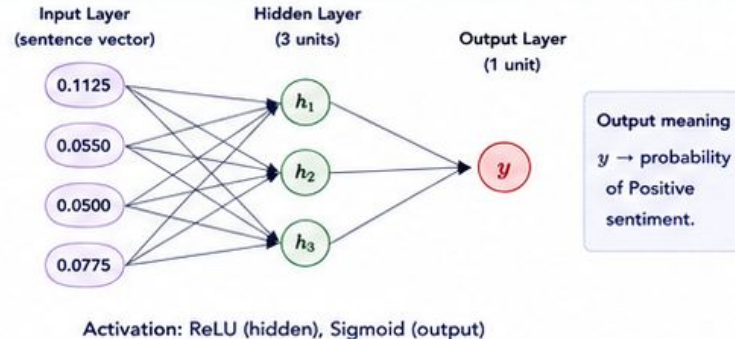
Sentence vector (4D)

0.1125 0.0550 0.0500 0.0775

Calculation example for  $d_1$ :

$$\frac{(0.21 + 0.31 + 0.05 - 0.12)}{4} = 0.1125$$

### 4. NEURAL NETWORK (SIMPLE MODEL)



### 5. FORWARD PASS (Compute score)

① Hidden layer ( $z = Wx + b$ , then ReLU)

| Unit  | $z$ (before ReLU) | $h = \text{ReLU}(z)$ |
|-------|-------------------|----------------------|
| $h_1$ | 0.37              | 0.37                 |
| $h_2$ | -0.15             | 0.00                 |
| $h_3$ | 0.28              | 0.28                 |

② Output layer ( $y = \text{sigmoid}(v^T h + c)$ )

Score ( $z_{\text{out}}$ ) = 0.46 →  $y = \text{sigmoid}(0.46) = 0.613$

### 6. COMPUTE LOSS

True label: Positive = 1

Prediction ( $y$ ) = 0.613

Use Binary Cross-Entropy Loss:

$$L = -[y \log(y_{\text{pred}}) + (1 - y) \log(1 - y_{\text{pred}})]$$

$$L = -[1 \times \log(0.613) + 0 \times \log(0.387)]$$

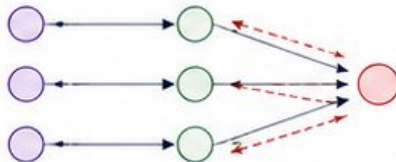
Loss  $L = 0.489$



Lower loss means prediction is closer to the true label.

### 7. BACKPROPAGATION (Compute gradients)

Error is sent backward to compute gradients for all weights and biases.



Gradients tell us how each parameter contributed to the error.

### 8. UPDATE PARAMETERS

Update weights and biases using Gradient Descent:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L$$

Where:

$\eta$  = learning rate (e.g., 0.1)

$\nabla L$  = gradient of the loss w.r.t. parameter  $\theta$



Parameters are updated to reduce the loss.

### 9. PREDICTION

After training for many epochs...

New prediction for "I love this movie!"

$y = 0.92$

Since  $y > 0.5$

Predicted Sentiment:

Positive 😊

### SUMMARY OF STEPS



### KEY TAKEAWAYS

- ✓ Word embeddings provide dense, meaningful inputs to the network.
- ✓ The network learns to map these inputs to the correct sentiment.
- ✓ Training minimizes loss; lower loss → better predictions.
- ✓ This same process works for other NLP tasks too!



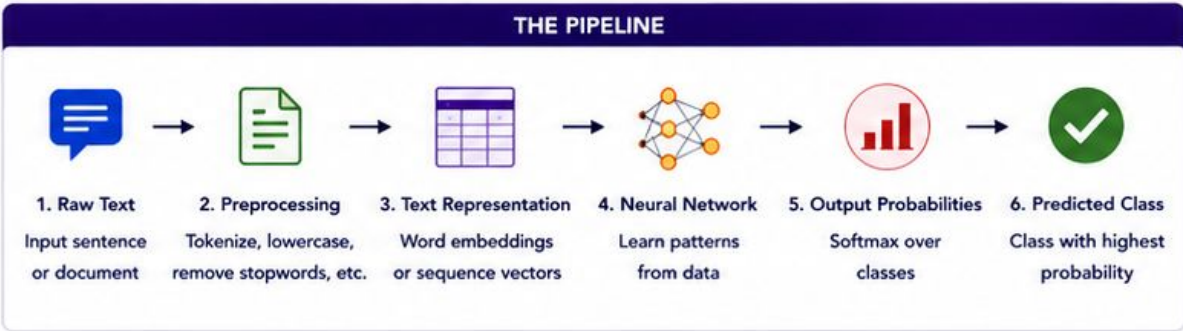
**Goal:**  
Accurate sentiment predictions on new, unseen sentences.



# Neural Network for Text Classification



Text classification assigns a label (e.g., **sentiment**, **topic**, **intent**) to a piece of text.  
A typical neural network approach:  
**Text → Representations → Neural Network → Probabilities → Predicted Class**



### 1. TEXT REPRESENTATION

Words are converted to dense vectors (embeddings) that capture meaning.

Example vocabulary (dim = 5)

| Word  | Embedding Vector                   |
|-------|------------------------------------|
| i     | [ 0.21, -0.11, 0.09, 0.05, 0.12 ]  |
| love  | [ 0.31, 0.23, -0.10, 0.17, -0.05 ] |
| this  | [ 0.05, -0.20, 0.26, -0.13, 0.08 ] |
| movie | [ -0.12, 0.30, -0.05, 0.22, 0.16 ] |
| !     | [ 0.00, 0.00, 0.00, 0.00, 0.00 ]   |

Sentence: "I love this movie!"  
Sequence of embeddings (one per token)

i      love      this      movie      !

Shape: (sequence\_length, embedding\_dim)

### 2. MODEL ARCHITECTURE (Example)

A simple feed-forward network on top of the sentence representation.

- Pooling summarizes the sequence into a single vector.
- Hidden layer learns useful features.
- Output layer gives probability for each class.

### 3. OUTPUT & PREDICTION

Softmax converts scores into probabilities. The class with the highest probability is the prediction.

Example (Sentiment Classification)

| Class    | Probability | Meaning            |
|----------|-------------|--------------------|
| Positive | 0.86        | Positive sentiment |
| Neutral  | 0.10        | Neutral sentiment  |
| Negative | 0.04        | Negative sentiment |

**Predicted Class: Positive** 😊

### 4. TRAINING (What happens)

- 1 Forward pass: compute prediction and loss (e.g., cross-entropy).
- 2 Backpropagation: compute gradients.
- 3 Update weights using Gradient Descent (or Adam).
- 4 Repeat for many batches and epochs until the model performs well.

Loss

Epochs

### 5. WORKED EXAMPLE

| Step 1: Input Text               | Step 2: Tokens      | Step 3: Embeddings (each is 5-dimensional) | Step 4: Pooling (Average) | Step 5: Network Output (Softmax Probabilities)               | Step 6: Prediction |
|----------------------------------|---------------------|--------------------------------------------|---------------------------|--------------------------------------------------------------|--------------------|
| I love this movie!<br>(Positive) | i love this movie ! | Shape: (5, 5)                              | Shape: (1, 5)             | 0.86 0.10 0.04<br>Positive Neutral Negative<br>Shape: (1, 3) | Positive 😊         |

The model learns from many such examples to generalize to new, unseen texts.

### TYPES OF MODELS (Examples)

- ✓ **Feed-Forward NN + Pooling**  
Simple and fast for short texts.
- ✓ **CNN for Text**  
Captures local patterns (n-grams).
- ✓ **RNN / LSTM / GRU**  
Good for sequences and long texts.
- ✓ **Transformer (e.g., BERT)**  
State-of-the-art for most NLP tasks.

### KEY TAKEAWAYS

- ✓ Texts are converted to dense vector representations.
- ✓ Neural networks learn patterns that help distinguish classes.
- ✓ Training minimizes loss; the model improves over time.
- ✓ Softmax gives probabilities; highest probability = prediction.
- ✓ This framework works for sentiment, topic, intent, spam detection, and many more NLP tasks.

**Goal: Build a model that correctly classifies the sentiment (or category) of new, unseen text!**

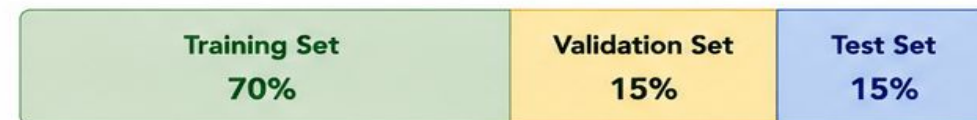


# Training, Validation and Test Data



To build a model that generalizes well to new, unseen data, we split our dataset into **three parts** with different purposes.

## THE DATA SPLIT (Typical)



The model never sees the Test set during training.

### 1. TRAINING DATA



Used to **train** the model.

- The model learns patterns and adjusts its weights.
- The goal is to minimize the training loss.



**Analogy:** Practice questions used while learning.

**Typical size:** Largest portion (e.g., 60–80%)

### 2. VALIDATION DATA



Used to **tune and make decisions**.

- Helps choose **hyperparameters** (e.g., learning rate, number of layers).
- Used for **early stopping** to prevent overfitting.



**Analogy:** Practice test to check understanding and adjust study strategy.

**Typical size:** Medium (e.g., 10–20%)

### 3. TEST DATA



Used to **evaluate final performance**.

- The model is **NOT** trained or tuned on this data.
- Gives an **unbiased estimate** of how the model performs in the real world.



**Analogy:** Final exam you take only once.

**Typical size:** Medium (e.g., 10–20%)

### EXAMPLE: SENTIMENT CLASSIFICATION

Dataset of 10,000 movie reviews

|                |                     |                                                     |
|----------------|---------------------|-----------------------------------------------------|
| Training set   | 7,000 reviews (70%) | Model learns patterns (positive, negative, neutral) |
| Validation set | 1,500 reviews (15%) | Tune hyperparameters, use early stopping            |
| Test set       | 1,500 reviews (15%) | Final evaluation of model (not seen before)         |



Example metrics on Test set:

**Accuracy = 0.87, F1-score = 0.86**

### WHY SEPARATE?



#### Avoid Overfitting

If the model sees the test data during training, it may just memorize it.



#### Fair Comparison

Different models can be compared fairly using the same test set.



#### Real-World Performance

The test set simulates new, unseen data the model will face in production.

### BEST PRACTICES

- ✓ Shuffle data before splitting.
- ✓ Keep the test set untouched until the very end.
- ✓ Use stratified splitting for classification tasks (to keep class distribution similar).
- ✓ If data is time-based, split chronologically (train → validation → test).
- ✓ Ensure data leakage does not occur (e.g., same user in train and test).



**Good data splitting leads to reliable models that perform well on real-world data!**

### THE JOURNEY



#### Raw Data

Collect all available data



#### Split Data

Train / Validation / Test



#### Train Model

Learn from training data



#### Tune & Validate

Adjust using validation data



#### Evaluate

Check final performance on test data



#### Deploy

Use the model on new data!



# Epochs and Batch Size



Two key training hyperparameters that control how the model learns from data.

## AT A GLANCE



### Epoch

One complete pass through the entire training dataset.



### Batch Size

Number of training examples used to compute the gradient and update the model once.

## 1. EPOCHS

- An epoch is when the model sees every training example once.
- The model makes many updates within an epoch (one per batch).
- More epochs = more opportunities to learn.

### EXAMPLE: 1 EPOCH

Dataset with 12 examples



1 epoch = 1 complete pass over all 12 examples



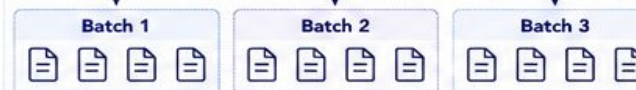
If we train for 5 epochs, the model will see the entire dataset 5 times.

## 2. BATCH SIZE

- Data is processed in small groups called batches.
- Each batch is used to compute the loss and update the model.
- Smaller batches → more updates per epoch.
- Larger batches → fewer updates per epoch.

### EXAMPLE: BATCHING (Batch Size = 4)

Dataset with 12 examples



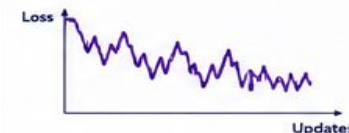
Steps (updates) in 1 epoch = 3

$$\text{In general: Steps per epoch} = \left\lceil \frac{\text{Number of examples}}{\text{Batch size}} \right\rceil$$

## 3. EFFECTS OF BATCH SIZE

### Small Batch Size (e.g., 16, 32)

- ✓ More updates per epoch
- ✓ Noisier gradients
- ✓ Better generalization
- ✓ More computationally expensive per epoch



Noisier but can escape sharp minima

### Large Batch Size (e.g., 256, 512)

- ✓ Fewer updates per epoch
- ✓ Smoother gradients
- ✓ Faster training per epoch
- ✓ May converge to sharper minimum solutions



Smoother but may get stuck in sharp minima



There is no single best batch size → it depends on your data, model, and hardware (memory).

## 4. HOW EPOCHS AND BATCH SIZE WORK TOGETHER

Training for 2 epochs with batch size = 4 on 12 examples



- ✓ 3 updates per epoch
- ✓ 6 updates in total (2 epochs × 3)
- ✓ More epochs = more learning
- ✓ Batch size controls how many examples are used per update

## 5. HOW TO CHOOSE?

| Goal                      | Suggested Choice                     |
|---------------------------|--------------------------------------|
| Faster training per epoch | Larger batch size                    |
| Better generalization     | Smaller batch size                   |
| Stable but efficient      | Medium batch size (typically 32–128) |
| More learning             | More epochs (with early stopping!)   |

Use validation loss and early stopping to decide the right number of epochs.

## 6. KEY TAKEAWAYS

- ✓ Epoch: one full pass over the entire training data.
- ✓ Batch Size: number of examples used to compute one update.
- ✓ Steps per epoch =  $\text{ceil}(\text{number of examples} / \text{batch size})$ .
- ✓ Smaller batch → noisier updates, better generalization.
- ✓ Larger batch → smoother updates, faster per epoch.
- ✓ Use validation set + early stopping to choose the right epochs.



Good combinations of epochs and batch size lead to faster convergence and better performance!

## QUICK FORMULA

$$\text{Steps per epoch} = \left\lceil \frac{\text{Number of training examples}}{\text{Batch size}} \right\rceil$$

Example:

10,000 examples, batch size = 100  
Steps per epoch =  $\text{ceil}(10000 / 100) = 100$

If you train for 10 epochs:

Total updates =  $100 \times 10 = 1,000$  updates



Data



Better model



# Underfitting vs Overfitting



The goal is to find a model that generalizes well to new, unseen data.

**Underfitting:** model is too simple to learn the patterns.

**Overfitting:** model is too complex and learns the noise.

## THE BIG PICTURE



Data



Model



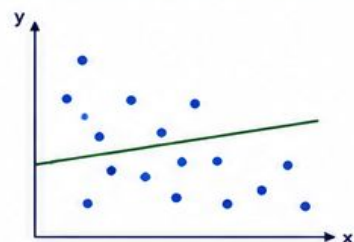
Evaluate



Good  
Generalization

We want low error on both training data and new (unseen) data.

## 1. UNDERFITTING (High Bias)



- ✓ Model is too simple.
- ✓ Fails to capture the underlying pattern.
- ✓ High error on training data and test data.

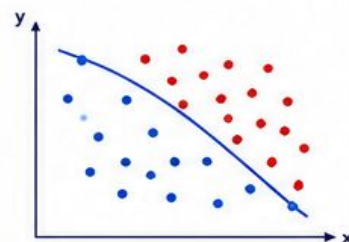
### Example (Classification)

A straight line trying to separate classes with a complex boundary.

### Typical Causes

- Model is too simple (e.g., too few layers/parameters)
- Not enough training (too few epochs)
- Excessive regularization
- Important features missing

## 2. GOOD FIT (Sweet Spot)



- ✓ Model captures the underlying pattern.
- ✓ Low error on training data.
- ✓ Low error on test data.

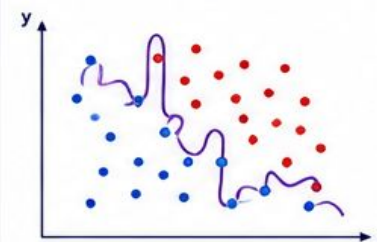
### Example (Classification)

A curved boundary that separates classes well and generalizes.

### What It Gives You

- Best generalization to unseen data
- Better real-world performance
- Balanced bias and variance

## 3. OVERFITTING (High Variance)



- ✓ Model is too complex.
- ✓ Learns noise in the training data.
- ✓ Low error on training data but high error on test data.

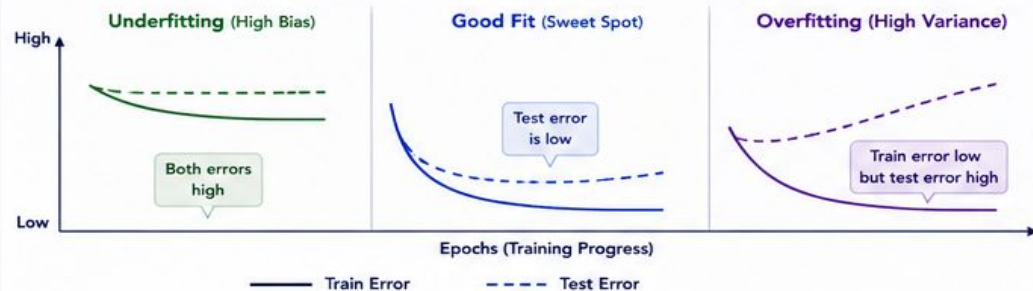
### Example (Classification)

A very wiggly boundary that fits even the noise (outliers).

### Typical Causes

- Model is too complex (e.g., many layers/parameters)
- Too many training epochs
- Too little data
- Insufficient regularization or dropout

## 4. TRAINING vs TEST ERROR CURVES



## 5. HOW TO ADDRESS

### If You See Underfitting

- ✓ Use a more complex model
- ✓ Train for more epochs
- ✓ Reduce strong regularization
- ✓ Add more relevant features
- ✓ Decrease dropout

### If You See Overfitting

- ✓ Use a simpler model
- ✓ Collect more training data
- ✓ Apply regularization (L2, dropout)
- ✓ Use early stopping
- ✓ Data augmentation

## 6. KEY TAKEAWAYS



**Underfitting = too simple**  
(High bias)  
Both training and test errors are high.



**Good fit = just right**  
Low training error and low test error.



**Overfitting = too complex**  
(High variance)  
Low training error but high test error.



Always aim for the **sweet spot** that gives the best performance on unseen (test) data — that's true generalization!



# How Can We Reduce Overfitting?



Overfitting happens when a model learns noise in the training data.

**Goal:** Improve generalization to perform well on unseen data.

Use one or more of the following strategies.

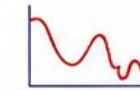
## THE BIG PICTURE



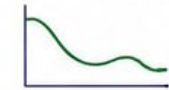
Data



Model



**Overfitting**  
(High variance)



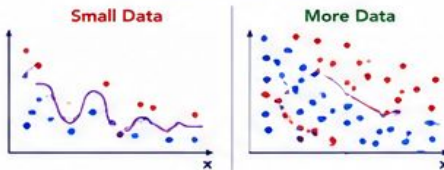
**Better Generalization**  
(Low test error)

Reduce model complexity and encourage learning of true patterns, not noise.

### 1. GET MORE DATA



More diverse data helps the model see more real-world patterns and less noise.



More data → better generalization

### 2. SIMPLIFY THE MODEL



Use a smaller model or fewer layers/parameters.

**Complex Model**



**Simpler Model**



Less capacity → less chance to memorize noise

### 3. REGULARIZATION (L2 / L1)



Add a penalty to large weights so the model stays simpler.

**L2 Regularization (Weight Decay)**

$$Loss_{total} = Loss + \lambda \sum_i w_i^2$$

**L1 Regularization**

$$Loss_{total} = Loss + \lambda \sum_i |w_i|$$

Smaller weights → simpler model

### 4. DROPOUT



Randomly drop (set to zero) some neurons during training.

**Without Dropout**



**With Dropout**

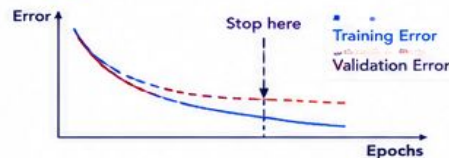


Prevents co-adaptation of neurons

### 5. EARLY STOPPING



Stop training when performance on validation set stops improving.



Prevents overfitting with more epochs

### 6. DATA AUGMENTATION



Create new training examples by transforming existing ones.

**Original sentence**

The movie was great.

**Synonym replace**

The film was fantastic.

**Random swap**

Great was the movie.

**Back translation**

The movie was great.

More variety → model learns robust patterns

### 7. BATCH NORMALIZATION



Normalize activations in each batch to stabilize and regularize training.

#### Benefits

- ✓ Reduces internal covariate shift
- ✓ Allows higher learning rates
- ✓ Acts as a regularizer

Stabilizes learning and reduces overfitting

### 8. REDUCE FEATURE COMPLEXITY



Use fewer or simpler features / embeddings.

**High-Dimensional (Noisy)**



**Lower-Dimensional (Cleaner)**



Less noise → better generalization

### EFFECT OF TECHNIQUES

| Technique                 | Reduces  | Improves           | Trade-off / Notes               |
|---------------------------|----------|--------------------|---------------------------------|
| More Data                 | Variance | Generalization     | May be costly/time-consuming    |
| Simpler Model             | Variance | Generalization     | May underfit if too simple      |
| Regularization (L1/L2)    | Variance | Generalization     | Choose $\lambda$ carefully      |
| Dropout                   | Variance | Generalization     | Too high dropout → underfitting |
| Early Stopping            | Variance | Generalization     | Needs validation set            |
| Data Augmentation         | Variance | Generalization     | Must preserve label meaning     |
| Batch Normalization       | Variance | Training Stability | Slight overhead                 |
| Reduce Feature Complexity | Variance | Generalization     | May lose useful information     |

### REMEMBER



Overfitting = high training performance but poor test performance.



Balance bias and variance to find the sweet spot.



No single technique is a magic bullet. Combine multiple strategies.



Always evaluate on validation/test data and monitor generalization.

### QUICK CHECKLIST

- ✓ Do you have enough (and diverse) training data?
- ✓ Is your model as simple as it can be while still capturing patterns?
- ✓ Have you applied regularization (L1/L2)?
- ✓ Are you using dropout?
- ✓ Are you using early stopping with a validation set?
- ✓ Can you augment your data?
- ✓ Are you using batch normalization (where appropriate)?
- ✓ Are your features/inputs necessary and not overly complex?



The goal is not to fit the training data, but to generalize to new, unseen data.

**Good generalization = low training error and low test error.**



# Dropout — Basic Concept



## What is Dropout?

Dropout is a **regularization technique** for neural networks that randomly "drops" (sets to zero) a number of neurons during training, preventing co-adaptation and reducing overfitting.

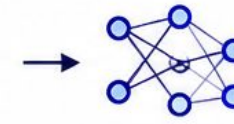
## THE IDEA



Train with all neurons



Randomly drop neurons ( $p = 0.5$ )



Network becomes sparser and more robust



Goal:

Better generalization and less overfitting.

## HOW DROPOUT WORKS

### 1. Keep Probability

Each neuron is kept with probability  $(1 - p)$  and dropped with probability  $p$ .



### 2. Apply During Training

Randomly set neuron outputs to 0 during each training iteration.

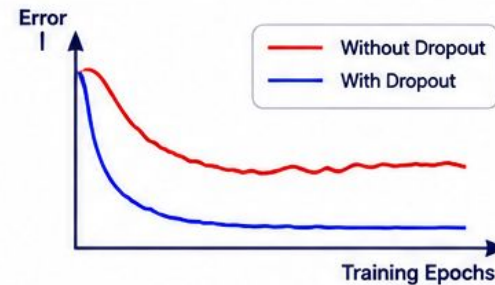


### 3. Scale at Test

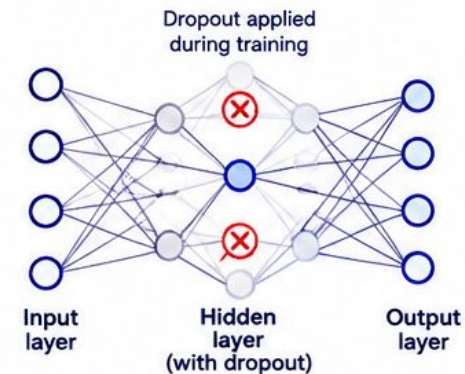
Use the full network at test time with outputs scaled by  $(1 - p)$ .



## EXAMPLE (REGULARIZATION EFFECT)



## NEURAL NETWORK WITH DROPOUT



## KEY BENEFITS



### Reduces overfitting

Prevents the model from memorizing training data.



### Improves generalization

Works better on unseen data.



### Acts as model averaging

Multiple thinned networks are learned.

## TRAINING vs TEST BEHAVIOUR

### Without Dropout

- ✗ Low training error
- ✗ High test error
- ✗ Overfitting

### With Dropout

- ✓ Slightly higher training error
- ✓ Lower test error
- ✓ Better generalization

## TYPICAL VALUES

| Task / Model           | Dropout Rate ( $p$ ) |
|------------------------|----------------------|
| NLP (e.g., BERT, LSTM) | 0.1 – 0.3            |
| CNN / Vision           | 0.3 – 0.5            |
| Small datasets         | 0.2 – 0.5            |



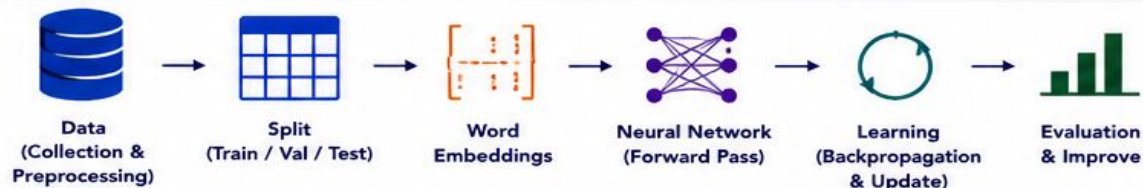
**Key Takeaway:** Dropout randomly disables neurons during training, making the model less dependent on specific patterns and leading to better generalization on unseen data.



# WEEK 4 – FINAL RECAP

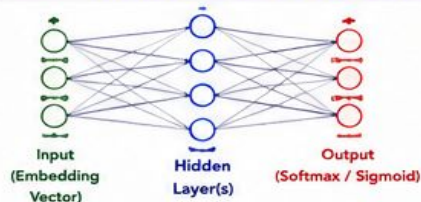


This week we covered the complete workflow of building and training neural networks for text classification — from data preparation and model architecture to learning, evaluation, and techniques to avoid overfitting for better generalization.



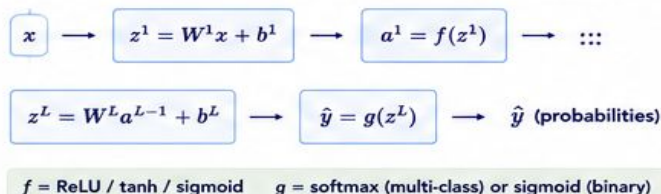
Goal: Low training error + Low test error = **Good generalization**

## 1. NEURAL NETWORK FOR TEXT CLASSIFICATION



- Input: dense word/document embeddings.
- Hidden layer(s): learn complex patterns.
- Output layer: produces probabilities for each class.

## 2. FORWARD PASS (PREDICTION)



## 3. LOSS FUNCTION (MEASURING ERROR)

Binary Classification (Binary Cross-Entropy)

$$L = -\frac{1}{N} \sum_{i=1}^N \left[ y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Multi-Class Classification (Categorical Cross-Entropy)

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Lower loss  $\rightarrow$  better model

## 4. BACKPROPAGATION (LEARNING SIGNAL)



### Key Idea

Use the chain rule to efficiently compute how much each weight and bias contributes to the final error.

## 5. UPDATE PARAMETERS (GRADIENT DESCENT)

Update rule:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W} \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

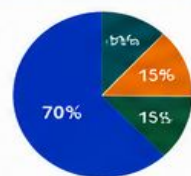
$\eta$  = learning rate

- Repeat for many batches/epochs until convergence.

## 6. EPOCHS AND BATCH SIZE

- Epoch: one complete pass over the entire training dataset.
- Batch size: number of training examples used to compute gradients and update the model.
- Small batch: noisy updates, more generalization.
- Large batch: stable updates, faster but may generalize less.
- More epochs: more learning, but risk of overfitting.

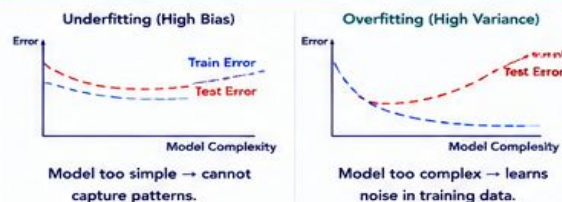
## 7. TRAINING, VALIDATION & TEST DATA



- 70% Training: Model learns patterns
- 15% Validation: Tune hyperparameters, early stopping
- 15% Test: Final unbiased evaluation

Good split prevents biased results and overfitting.

## 8. UNDERFITTING vs OVERFITTING



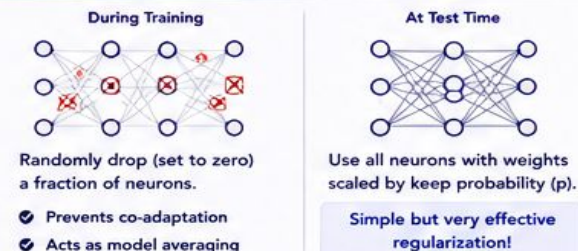
Goal: find the sweet spot (good bias-variance trade-off).

## 9. HOW CAN WE REDUCE OVERFITTING?

- Get more data
- Simplify the model
- Use regularization (L2/L1)
- Apply dropout
- Use early stopping
- Data augmentation
- Batch normalization
- Reduce feature complexity / use better features

Combine multiple techniques for best results.

## 10. DROPOUT – BASIC CONCEPT



## KEY TAKEAWAYS

- Neural networks process text representations to learn discriminative patterns for classification.
- Learning happens through forward pass, loss computation, backpropagation, and parameter updates.
- Proper data splits, training settings, and regularization are essential for good generalization.
- Always monitor performance on validation data and evaluate once on unseen test data.
- Practice, experiment, and small improvements compound into strong NLP models!

## REMEMBER

- No single trick is magic. Understand the basics and combine methods.
- Monitor Train & Val curves. They tell you if you are underfitting or overfitting.
- Generalization is the goal: perform well on new, unseen data.
- Iterate  $\rightarrow$  Evaluate  $\rightarrow$  Improve  $\rightarrow$  Repeat!



The ultimate goal is not to fit the training data, but to **generalize** and perform well on **NEW**, unseen data.